In this course, we will build a practical toolkit for mastering Codex and taking confident control of AI-driven development workflows. We will examine why some workflows improve with continued use while others plateau, then learn how to recognize and correct the patterns that hold us back.

What we will learn

We will develop the judgment needed to work with Codex deliberately rather than relying on trial and error. By the end of the course, we will be able to:

- Recognize the thought processes, mental models, and common mistakes that determine whether a Codex workflow improves over time.
- Write prompts that consistently produce accurate results by understanding what to specify and what to leave flexible.
- Make intentional decisions about session context, including when to keep it compact, resume an earlier session, start fresh, maintain focus, or allow exploration.
- Understand the cost of poor context and session decisions before they lead to wasted time or repeated corrections.
- Turn AGENTS.md from passive documentation into an active set of instructions that guides Codex.
- Scale a shared memory and instruction system across a development team so everyone benefits from consistent guidance.
- Review and verify AI-generated output carefully, including issues that automated tests can miss.
- Read diffs closely so small errors do not accumulate and reduce the quality of the project.
- Diagnose failed sessions, choose the right workflow tool for a task, and decide whether to reset or persist before entering a correction spiral.

Required Codex experience

This course builds on existing Codex knowledge rather than introducing Codex from the beginning. We should already be comfortable running sessions, writing prompts, using AGENTS.md, and working with Git to review or revert changes.

Familiarity with advanced Codex features will also help us follow the course. These features include sub-agents, parallel sessions, hooks, and skills. We do not need to be experts in every feature, but we should understand what they are and ideally have used some of them.

Terminal and technical setup

We should be comfortable navigating a terminal and using basic Git commands. The course will rely on operations such as git diff, git status, git commit, and git restore, so these should feel familiar.

Before continuing, we also need Node.js version 18 or higher, a correctly configured Codex installation, and either an OpenAI API key or a ChatGPT Pro or Max subscription. With these foundations in place, we will be ready to focus on applying Codex more effectively and accelerating our mastery throughout the course.

© 2026 Zenva Pty Ltd. All rights reserved.

Codex can be used in two fundamentally different ways. We can treat it as an executor that follows a prescribed recipe, or we can act as a director who defines the destination and the boundaries. The director mindset gives Codex room to reason about implementation while keeping us responsible for the outcome.

Executor and director mindsets

When we use Codex as an executor, we treat it like a fast typist. We provide function names, variable names, and a detailed sequence of operations, then ask it to translate those decisions into code. This can be useful when every implementation detail is already known, but it also limits Codex to the path we selected.

A director starts from a different question: **What should exist when this task is complete?** We describe the user-facing result, identify the rules that must be respected, and allow Codex to determine an appropriate implementation. We then evaluate whether the proposed solution satisfies the outcome and constraints.

- **Core question:** An executor asks how to implement a step, while a director asks what should be true when the work is complete.
- **Prompt focus:** An executor specifies implementation details, while a director specifies outcomes, rules, and boundaries.
- **Approach:** An executor pre-decides the sequence, while a director leaves room for Codex to choose a suitable path.
- **Result:** An executor receives exactly the requested implementation, while a director can benefit from alternatives that are more efficient, robust, or consistent with the existing project.

Comparing two prompts for the same task

Consider an invoice form that must validate input, save an invoice, and respond appropriately to success or failure. An executor-oriented prompt might prescribe a specific request and promise chain:

```
fetch('/api/invoices', {
  method: 'POST',
  body: JSON.stringify(Object.fromEntries(formData)),
  headers: {
    'Content-Type': 'application/json',
  },
})
  .then(response => response.json())
  .then(data => {
    console.log('Invoice saved:', data)
    // router.push('/invoices')
  })
  .catch(error => console.error('Error saving invoice:', error))
```

This prompt has already chosen the API endpoint, request method, data conversion, response handling, and redirect strategy. Codex can implement those instructions, but it lacks freedom to inspect the project and choose a better existing pattern. Any weakness in the prescribed design is carried into the result.

A director-oriented prompt describes the required behavior and the important constraints instead:

© 2026 Zenva Pty Ltd. All rights reserved.

```
When the user submits the invoice form, validate the required fields.
If the form is valid, save the invoice and redirect to the invoice list.
If validation fails, show errors inline without losing the form data.

Use the existing createInvoiceAction from invoiceActions.js.
Do not add any new dependencies.
```

This version still provides clear direction. It defines successful behavior, failure behavior, an existing project pattern to reuse, and a dependency boundary. However, it does not dictate every implementation step, so Codex can inspect the surrounding code and propose a solution that fits the codebase.

What changes when we act as a director

The shift is not about making prompts longer. It changes what we delegate and how we review the result.

1. **Define the outcome.** Describe success in terms of what the user or system must be able to do.
2. **Set the constraints.** Identify the technology, established patterns, required files, forbidden changes, compatibility needs, and other non-negotiable rules.
3. **Evaluate the proposal.** Review Codex's approach and accept, reject, or refine it based on the desired outcome.
4. **Review for effectiveness.** Check whether the solution works well and respects the constraints, not merely whether it followed a predetermined sequence.

We remain the architectural decision maker and quality gate. Codex chooses an implementation path within the space we define, while we remain accountable for verifying the result.

Recognizing an executor prompt

Specific function names, variable names, and sequence words such as then or next can reveal that we have pre-decided the implementation. When these details appear, we should pause and ask whether they are genuine constraints or decisions that Codex could make after inspecting the project.

A strong director prompt usually contains a user-facing outcome, clear boundaries, and enough project context to support good decisions. It leaves implementation open where flexibility is safe, then requires us to inspect and verify the completed work. This balance uses Codex as a reasoning partner without giving up control over quality.

© 2026 Zenva Pty Ltd. All rights reserved.

Effective AI prompting requires a balance between useful direction and excessive control. If we provide too little information, Codex must fill important gaps with assumptions. If we dictate every implementation detail, Codex cannot apply its knowledge of the existing codebase. In this lesson, we will learn how to find the middle ground by describing outcomes, setting genuine constraints, and deliberately leaving implementation choices open.

The two failure modes

Prompts commonly fail in one of two ways:

- **Underspecification:** The prompt does not provide enough detail, so Codex makes plausible but potentially incorrect assumptions about what we need.
- **Overspecification:** The prompt dictates every step, preventing Codex from adapting its solution to the project's architecture, patterns, and naming conventions.

Both approaches create a hidden cost. Vague prompts lead to correction cycles as we untangle incorrect assumptions. Overly prescriptive prompts lead to re-engineering when the requested implementation does not fit the codebase well.

Why vague prompts create rework

Consider the prompt, "Add filtering to the invoice list." Although it sounds clear, it leaves several important decisions unanswered. Codex must choose which fields can be filtered, how the interface should look, whether filtering belongs on the client or server, and how the application should manage filter state.

Each decision might be reasonable in isolation, but it can still conflict with the project's established patterns or our actual intent. We then have to explain each mismatch and guide Codex back toward the desired result. The problem is not that Codex cannot make decisions. The problem is that the prompt leaves business and user-experience decisions unstated.

Why excessive control also creates rework

At the other extreme, we might specify the exact component name, button layout, state variable, filtering method, property names, and CSS class before Codex has examined the project. For example, we could require an InvoiceFilterBar component, a particular state hook, a specific array filtering call, and an exact active class.

This approach gives Codex no room to discover that the codebase already uses another component pattern, naming convention, state-management approach, or server-side filtering system. Codex becomes a fast typist instead of a context-aware collaborator. A technically complete prompt can therefore produce a rigid and poorly integrated solution.

Finding the right level of specificity

A balanced prompt explains what users should be able to do and which boundaries the solution must respect. For the invoice example, we can ask for a filter bar that lets users filter by All, Draft, Sent, Paid, and Overdue. We can state that only one filter may be active at a time, the active choice must be visually highlighted, existing component and state-management patterns should be followed, and no new dependencies should be added.

This version is specific about the result and the non-negotiable requirements. It does not decide the component name, function names, file structure, or exact sequence of operations. Codex can inspect the existing project and choose an implementation that fits naturally.

© 2026 Zenva Pty Ltd. All rights reserved.

A reliable three-part prompt formula

We can build effective prompts from three components:

1. **Describe the outcome.** Explain the desired end state from the user's point of view. Focus on what should exist, work differently, or be experienced when the task is complete.
2. **Define the constraints.** State the non-negotiable rules that the solution must respect. These can include business rules, the technology stack, existing architectural patterns, dependency restrictions, and areas that must not be changed.
3. **Choose what to leave out.** Avoid prescribing function names, variable names, state-management details, file layouts, and operational sequences unless they are genuine requirements.

The third part is intentional, not accidental. Leaving out implementation details gives Codex the freedom to use the wider codebase context and select an efficient solution.

Put specificity into the outcome

We should be highly specific about areas where our knowledge matters most:

- The desired user experience
- Exact business rules
- Relevant edge cases
- Required compatibility or architectural boundaries
- Files, systems, or behaviors that must remain unchanged

We should usually avoid prescribing exact function names, variable names, file structure, or the order in which the work must be performed. Codex can handle these how-to decisions more effectively after inspecting the codebase. The exception is when an implementation choice is truly required by an existing pattern or external requirement. In that case, we should state it clearly as a constraint.

Test each sentence before sending

Before sending a prompt, we can review it sentence by sentence with three questions:

1. **Is this outcome-focused?** If the sentence describes what users should experience or what the completed state should be, keep it.
2. **Is this implementation-focused?** If the sentence describes how to build the solution, remove it or rewrite it as a constraint.
3. **Is this a genuine constraint?** If a specific implementation choice is necessary because of an established pattern or requirement, state that necessity clearly instead of presenting step-by-step instructions.

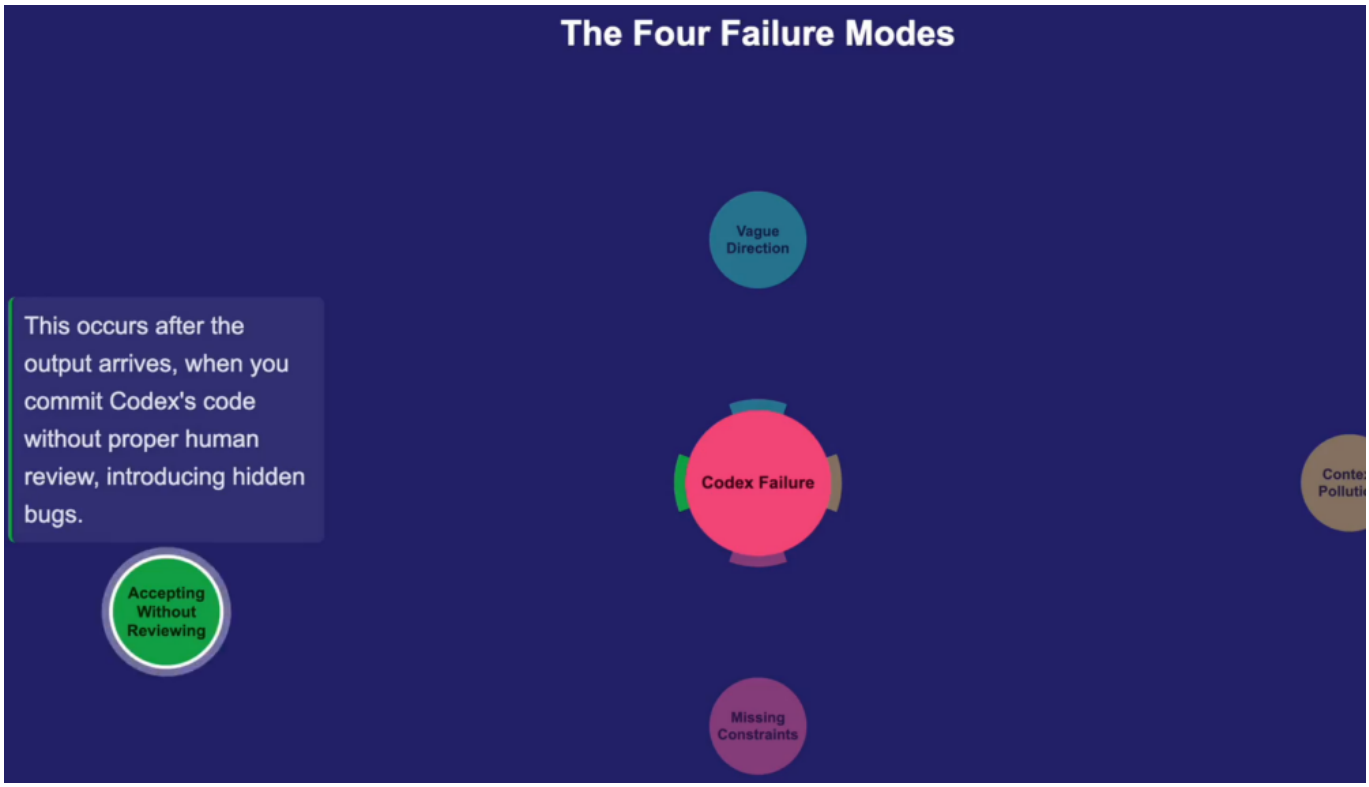
This practical test helps us avoid both failure modes. We give Codex enough direction to understand the desired result while preserving the autonomy it needs to make context-aware decisions. The goal is neither vagueness nor control. It is precise communication about outcomes and boundaries.

© 2026 Zenva Pty Ltd. All rights reserved.

Codex sessions do not all fail for the same reason. By separating failure into four distinct modes, we can identify the real problem, choose a targeted response, and avoid spending time on a fix that makes the session worse.

The four failure modes

The four fundamental failure modes are vague direction, context pollution, missing constraints, and accepting without reviewing. Each appears at a different stage of the workflow and produces recognizable symptoms.



- **Vague direction** begins before or at the start of a session. The desired outcome has not been defined clearly enough, so Codex fills the gaps with assumptions. Its output may be plausible, but generic or wrong in several ways.
- **Context pollution** develops during a long session. Irrelevant information from multiple tasks accumulates in the context window, and the output becomes inconsistent, contradictory, off-topic, or lower quality.
- **Missing constraints** occur inside the prompt or project guidance. Codex has enough information to produce working code, but it has not been told which libraries, patterns, or approaches are prohibited. The result can be technically correct while still violating project rules.
- **Accepting without reviewing** happens after Codex returns an answer. Committing generated changes without human review can introduce subtle logic errors, unexpected state changes, and bugs that only become visible later.

Fixing vague direction

When a session begins with an unclear brief, Codex must invent missing details. Additional steering messages in the same session rarely repair that foundation because they add more information to an already unclear direction.

The fastest fix is to start a new session and write a more specific prompt. We should state the

© 2026 Zenva Pty Ltd. All rights reserved.

intended outcome and provide all necessary project context from the beginning. A clean restart is more effective than trying to salvage an implementation based on incorrect assumptions.

Cleaning up context pollution

A session can begin well and degrade over time as unrelated discussions and tasks consume the context window. The important symptom is change over time: Codex was initially focused, but later responses become inconsistent or contradictory.

For continued work on the current task, we can use /compact to reduce the active context. For a different task or a completely fresh start, we can use /new. If the working state has become unusable, we can also return the repository to a known clean commit before continuing.

Making constraints explicit

Missing constraints produce output that works in isolation but does not fit the project. For example, Codex may select a common library or pattern that the codebase does not allow because no instruction ruled it out.

We should state the missing rule explicitly and ask Codex to redo the work within that boundary. We should then add the rule to AGENTS.md, turning a one-time correction into a persistent guardrail for future sessions.

Reviewing before accepting

AI-generated code should be reviewed with the same care as a peer's pull request. Before committing it, we need to inspect the diff, verify the behavior, and look for subtle changes that were not requested.

If an unexplained bug is traced to work committed several revisions ago, git bisect can help locate the breaking commit. Once found, we can revert the faulty change, review it carefully, and only reapply a corrected version.

Diagnosing the current failure

A short diagnostic sequence helps us match symptoms to causes:

1. If the output is plausible but wrong in several ways from the beginning, diagnose **vague direction**. Start over with a clear, specific brief.
2. If the session began well but its output became inconsistent or degraded over time, diagnose **context pollution**. Use /compact for the current task or /new for a fresh session.
3. If the code works but violates a project rule, diagnose **missing constraints**. State the rule, redo the work, and record the constraint in AGENTS.md.
4. If a mysterious bug came from an earlier unreviewed commit, diagnose **accepting without reviewing**. Locate and revert the bad change, then review it before recommitting.

Why the right diagnosis matters

Applying the wrong fix can compound the original problem. More steering messages add noise when the original direction was vague. Better prompts inside a polluted session consume even more context. Repairing individual rule violations without documenting the missing constraint guarantees that the problem can recur. Asking Codex to debug its own old, unreviewed output without the necessary context also leaves it poorly equipped to find the cause.

The central skill is diagnosis. When we recognize the failure mode first, we can apply the correct

© 2026 Zenva Pty Ltd. All rights reserved.

solution quickly, protect the codebase, and regain control of the session without unnecessary rework.

© 2026 Zenva Pty Ltd. All rights reserved.

Best Practices for Agentic Coding with Codex - Directing Codex Effectively

1. Codex must add invoice filtering. Which prompt best uses the director mindset?

a. Define the filter behavior and project constraints, then let Codex choose the implementation.
b. Specify every function name, state variable, and file edit.
c. Say only "add filtering" and provide no context.
d. Prevent Codex from inspecting the existing project.

2. Codex writes working code with a prohibited library. Which failure mode is this?

a. Vague direction
b. Context pollution
c. Missing constraints
d. Accepting without review

3. A session began well, but later answers became contradictory. What should you do for the same task?

a. Add more unrelated examples.
b. Use /compact to reduce polluted context.
c. Prescribe every implementation step.
d. Commit the latest result.

© 2026 Zenva Pty Ltd. All rights reserved.

Concrete examples give Codex a clearer target than abstract instructions. In this lesson, we will learn why examples reduce ambiguity, how to reference project files with the @ symbol, when to provide before-and-after data, and how to avoid examples that send Codex in the wrong direction.

Why examples outperform descriptions

A written description introduces two points where information can be lost. First, we translate our mental model into words. Details that seem obvious to us may be omitted, and some wording may be interpreted in more than one way. Codex must then translate those words back into code, creating another opportunity for the result to drift from our intent.

A direct file reference removes much of this lossy translation. Codex receives a real implementation from the project and can inspect its exact conventions. Instead of guessing what we mean by “use the same structure,” it can identify the structure for itself.

These details often determine whether generated code merely works or genuinely belongs in the project:

- Whether component props are destructured in the function signature or accessed later
- Whether files use named exports or default exports
- Where imports, including CSS module imports, are placed
- How files and CSS classes are named
- How JSX return statements are formatted

Describing every convention would make a prompt long and fragile. Referencing a representative file lets the existing code communicate those choices directly.

Reference files with the @ symbol

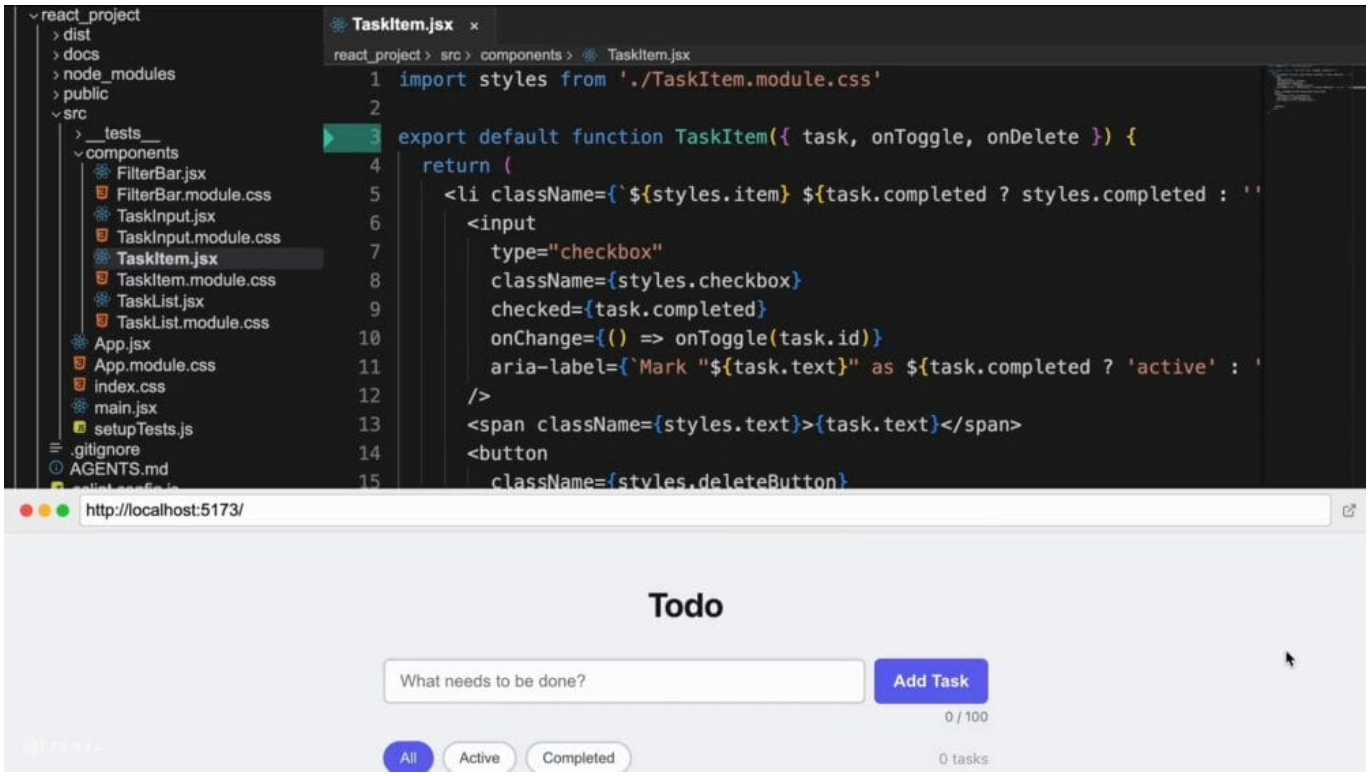
In a Codex prompt, type @ to open a file picker and select a file from the current project. The reference is not merely a link. Codex receives the file contents as part of the prompt context, including imports, function and class names, formatting, and other implementation details.

For example, an abstract prompt might ask for a new TaskNote component that follows the structure and naming conventions of existing components. That request still leaves many choices open. A more concrete prompt references @src/components/TaskItem.jsx and @src/components/TaskItem.module.css, then asks Codex to create TaskNote with the same file structure, CSS module pattern, class naming, and prop usage.

Apply a reference pattern in Codex

The Todo project contains a TaskItem.jsx component and a matching CSS module. The component provides a practical pattern for the new read-only note component, while the running application provides context for how the project currently behaves.

© 2026 Zenva Pty Ltd. All rights reserved.



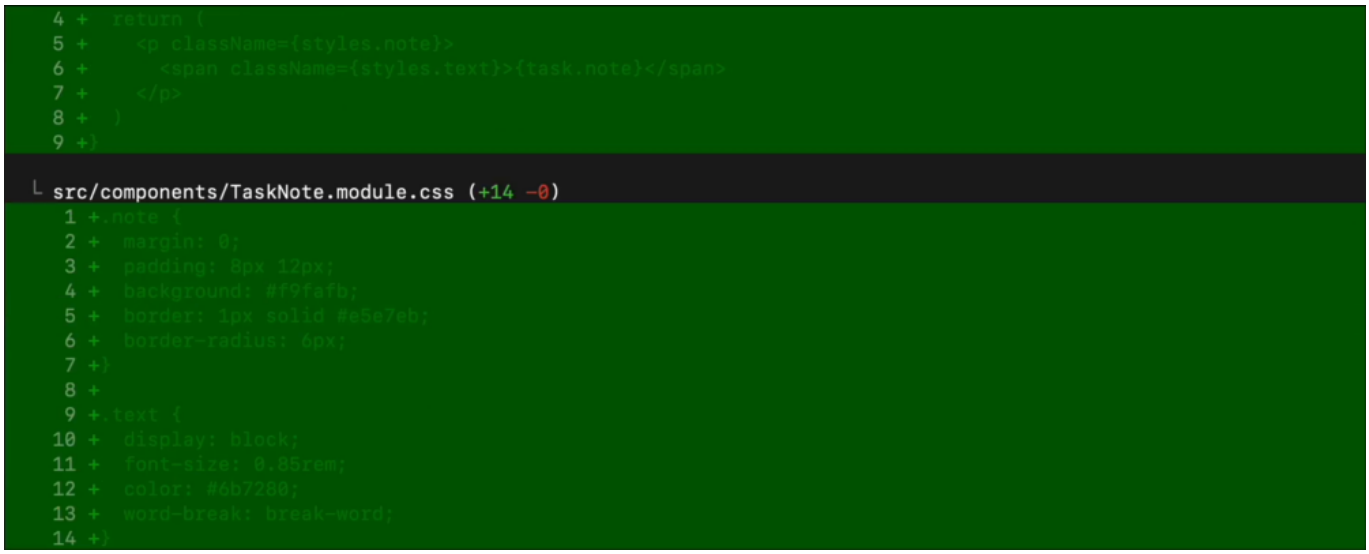
Before asking Codex to make changes, we initialize Git, add the existing files, and create a first commit. This establishes a clean baseline that makes later changes easy to inspect. We then launch Codex and enter a prompt that references both TaskItem files.

```
@src/components/TaskItem.jsx @src/components/TaskItem.modu

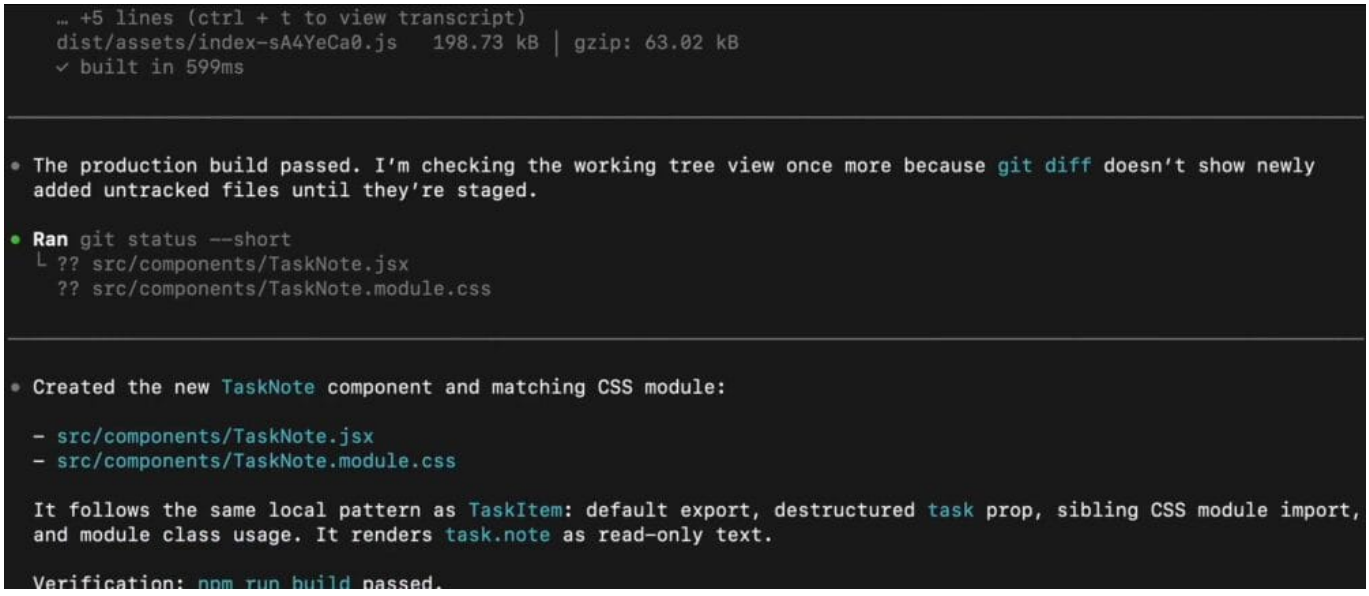
Create a new TaskNote component that displays a short
read-only note attached to a task. Follow exactly the
same structure and CSS module pattern as TaskItem —
same file naming, same class naming conventions, same
way of receiving and using props.
```

Codex reads the referenced component and stylesheet before making changes. It creates TaskNote.jsx and TaskNote.module.css, matching the established default export, destructured prop, sibling stylesheet import, file naming, and class naming patterns.

© 2026 Zenva Pty Ltd. All rights reserved.



Generation is only part of the workflow. We run the production build to verify that the new files integrate successfully, inspect Git status to confirm the expected files were created, and review Codex's summary of how the result follows the reference. Once the result is approved, Codex can stage and commit the files with a descriptive message.



Use file references for pattern matching

The same technique applies whenever a task should follow an existing project pattern:

- Reference an existing component when creating a component with the same structure and style.
- Reference an existing API route when a new route must follow the same routing and error-handling conventions.
- Reference an existing test when a new test should use the same setup and assertion style.
- Reference an existing service function when the new function must match the project's service shape and error handling.

The most useful reference is not simply a file of the same general type. It is the file whose role and complexity most closely match the requested result.

© 2026 Zenva Pty Ltd. All rights reserved.

Show before-and-after transformations

A complete file is useful for broad structural patterns, but focused transformation tasks often need a different kind of example. Show Codex a specific input and the exact output you want.

This approach works well when reformatting a date string, converting an array into an object, renaming fields while changing their values, or applying a custom sorting rule. The pair demonstrates the transformation directly, which is usually faster and more reliable than describing the logic in prose.

When multiple records require the same transformation, provide one representative input-and-output pair and state that the pattern should be applied to every record. Codex can then infer the rule from concrete evidence.

Avoid misleading examples

Examples are powerful because Codex follows them closely, but that also makes the choice of example important. Watch for two common problems:

- **Outdated examples:** A file from before a refactor may no longer represent the team’s conventions. Codex may reproduce the old pattern faithfully, leaving us with inconsistent code and manual corrections. Always choose a recent, representative file.
- **Contextually wrong examples:** A simple read-only display component may be a poor model for a complex form. Choose a reference that is both structurally and functionally relevant. If no suitable example exists, a clear and detailed description may be more effective than a misleading reference.

Follow the show-first rule

Whenever we start describing a pattern, we should ask whether we can show that pattern to Codex instead. If the pattern already exists in the codebase, reference the exact file with @. If the task transforms data, provide a concrete before-and-after pair. Reserve detailed descriptions for genuinely new concepts that have no useful existing example.

This show-first approach reduces ambiguity and gives Codex direct evidence of the conventions and results we expect.

© 2026 Zenva Pty Ltd. All rights reserved.

A strong prompt can be more than a one-time instruction. When it captures a reusable way to solve a recurring problem, it becomes an asset that can improve future Codex sessions. In this lesson, we will learn how to recognize prompts worth saving, choose the right storage method, organize each saved prompt, and build a capture habit that creates long-term value.

Recognizing prompts worth saving

The most important distinction is whether a prompt solved a **type of problem** or only one specific instance. A valuable prompt has a reusable structure that can work for similar tasks after we replace a few details. It gives us a template for future solutions rather than a command that only makes sense at one moment in one project.

We should save a prompt when we can reasonably expect to face the same kind of task again. Examples include adding a new feature that follows an established pattern or refactoring a common structure. A prompt tied to one unusual bug or one temporary project state is less useful as a library entry, so we can let those one-off commands go.

Signals of an effective prompt

Several signals can help us identify an unusually effective prompt:

- **First-attempt success:** Codex produced an accurate, complete result without a correction cycle because the prompt supplied the necessary context and constraints.
- **Minimal editing:** The output was close enough to our intended result that we could use it with little modification.
- **Sound judgment:** Codex handled nuanced decisions correctly and respected the constraints in situations where a less precise prompt could have led it in the wrong direction.
- **Future applicability:** We can imagine using the same prompt pattern for another task of the same type, even when the specific details change.

These signs show that the prompt did more than produce a satisfactory answer. It communicated our intent clearly enough to become a reliable starting point for future work.

Choosing where to save a prompt

The best storage method depends on the scope and purpose of the prompt:

- **Save a complete workflow as a skill.** If the prompt represents a multi-step process that benefits the team, we can store it as a skill under .codex/skills/. A skill can be invoked by name, accept inputs, and turn a proven workflow into a shared tool.
- **Add enduring rules to AGENTS.md.** If a prompt expresses a constraint or coding convention especially well, we can preserve the exact phrasing in the project guidance. This makes the rule part of the project’s lasting context and encourages consistent decisions.
- **Use a lightweight personal library.** For frequent prompts that are personal or specific to one project, a Markdown file, editor snippet, or running document provides a simple way to collect them without the overhead of a formal skill.

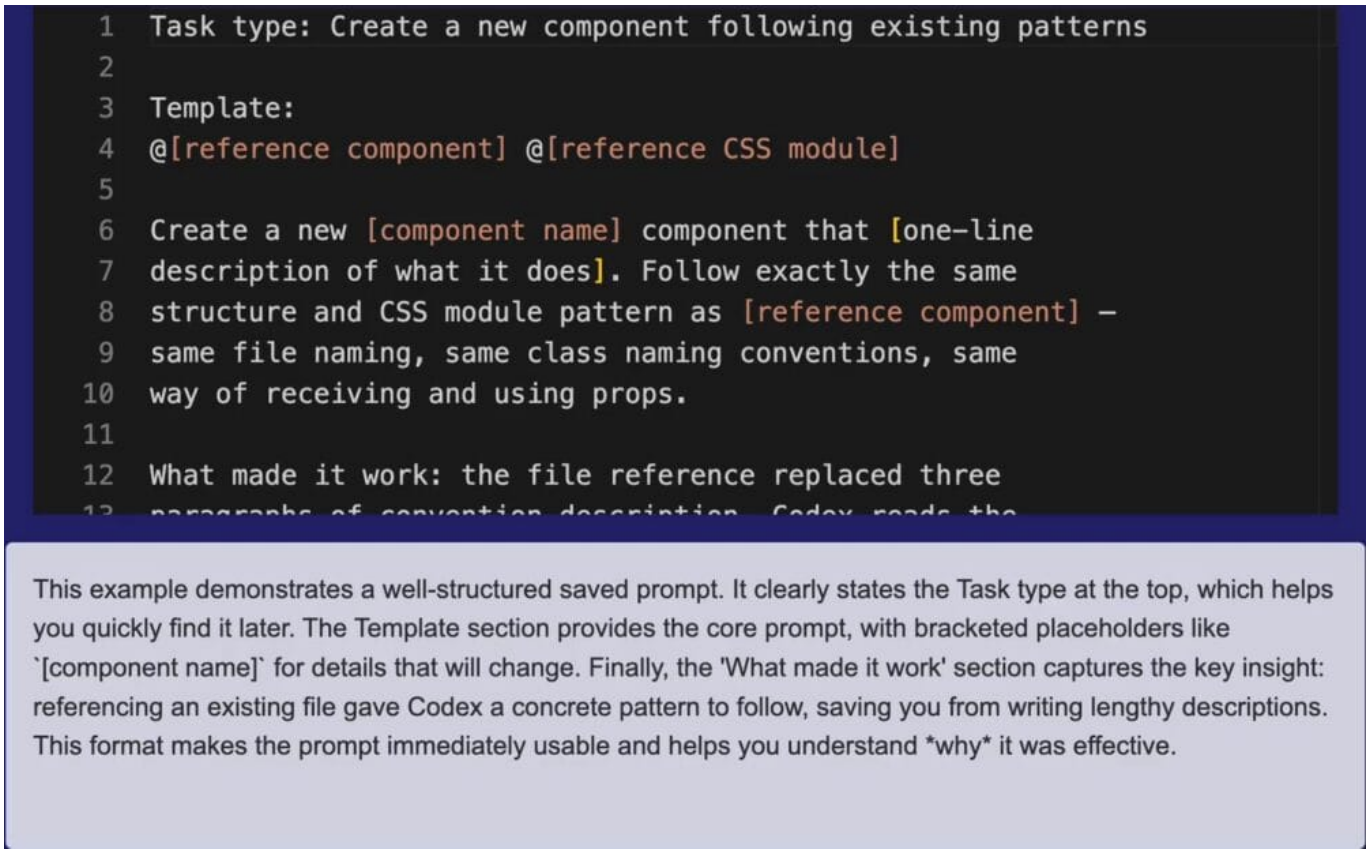
Structuring a saved prompt

A useful library entry should explain both what to run and why it works. We can organize it into three parts:

1. **Task type:** Give the entry a clear category so it is easy to find later.
2. **Template:** Preserve the core prompt and use bracketed placeholders for details that will change, such as the component name or a reference file.

© 2026 Zenva Pty Ltd. All rights reserved.

3. **What made it work:** Record the key insight that produced the successful result.

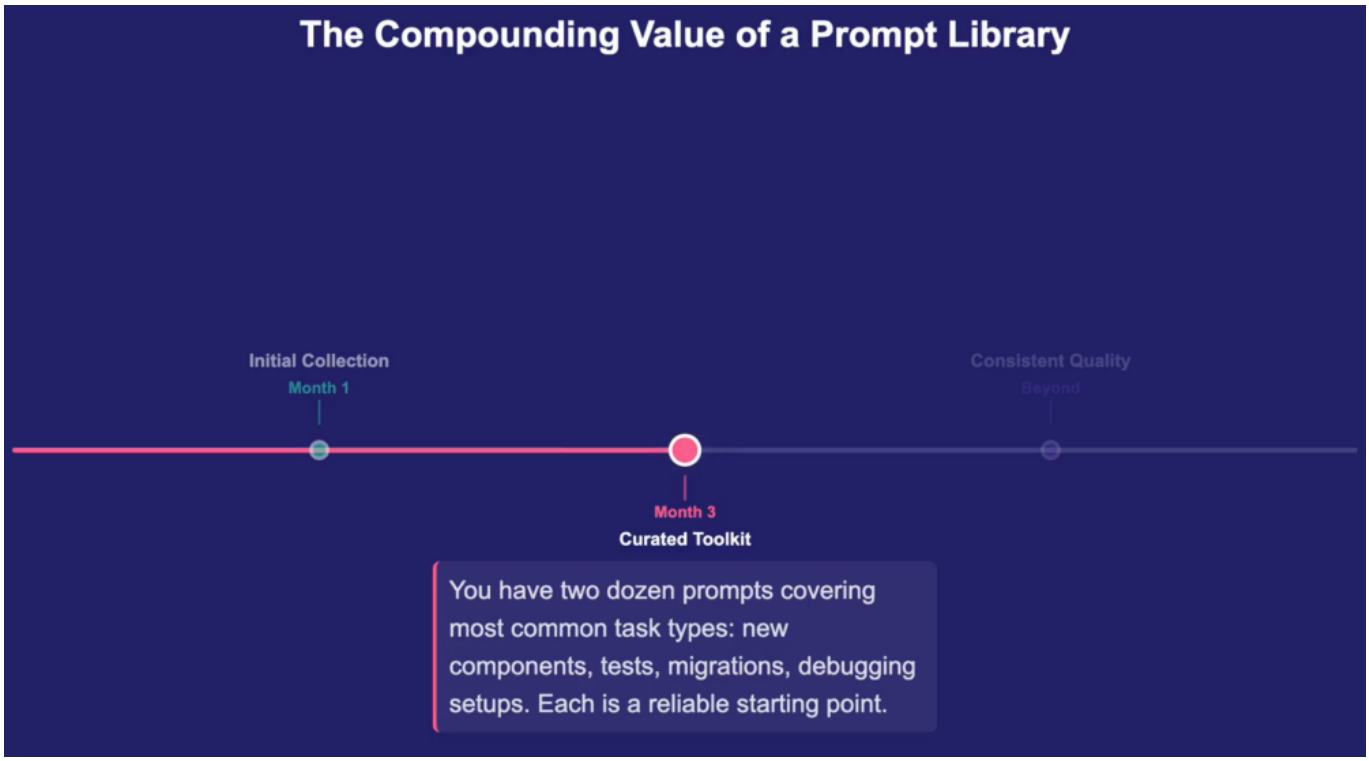


For example, a component-creation prompt can reference an existing component and its CSS module, then ask Codex to follow the same structure, naming conventions, and approach to props. The file references provide a concrete pattern that Codex can inspect directly, which avoids a lengthy and potentially inaccurate description of every convention. Recording this reasoning makes the prompt immediately reusable and helps us understand why it succeeded.

Building compounding value

A prompt library becomes more valuable as it grows. During the first month, a small initial collection may only save a few minutes at a time. By around month three, it can develop into a curated toolkit of roughly two dozen prompts that cover common task types, including new components, tests, migrations, and complex debugging setups.

© 2026 Zenva Pty Ltd. All rights reserved.



Each saved prompt provides a reliable starting point. As the library matures, results become more consistent, correction cycles shrink, and first-attempt output quality improves. Every new successful experience can enrich the collection further, so the value compounds over time.

Creating a capture habit

The best time to save a successful prompt is immediately after it works. At that moment, the exact prompt, its output, and the surrounding context are still fresh, which makes it easier to document the important details accurately.

Before closing a session, we can ask one simple question: **Did we write a prompt today that we would want to use again?** If the answer is yes, we should capture it then and there. Waiting until the next day can bury the prompt in session history, while waiting a week may make it difficult to recover at all. Spending two minutes to save it now can save twenty minutes when we encounter a similar task later.

© 2026 Zenva Pty Ltd. All rights reserved.

Best Practices for Agentic Coding with Codex - Reusable Prompt Patterns

1. A new React component must match existing project conventions. What should your prompt include?

a. The closest component and stylesheet referenced with @
b. A long generic description of clean code
c. An unrelated large file from the project
d. Only the desired component filename

2. You need Codex to convert many dates to one format. Which example is most useful?

a. One representative input and its exact expected output
b. Several paragraphs about good formatting
c. A file that uses outdated conventions
d. A finished result without its source input

3. Which prompt is most worth saving?

a. A recurring task pattern that produced a strong first result
b. A one-off command tied to a temporary project state
c. A vague prompt that required repeated corrections
d. An exact command using obsolete filenames

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will examine what fills a Codex context window, estimate the context cost of common development operations, monitor usage with /status, and decide when an operation should run outside the session.

Recognizing hidden context costs

Conversation messages are the most visible use of context, but they are only one part of the total footprint. Several other inputs and outputs also consume space:

- **Conversation messages:** Our prompts and Codex's responses become part of the active context.
- **Project instructions:** When an AGENTS.md file applies to the workspace, Codex loads its instructions at the start of the session. Keeping this file concise avoids unnecessary context usage.
- **Files:** The contents of files that Codex reads to understand or modify a project are added to the context. Large files, or several files read in succession, can add up quickly.
- **Tool and command output:** Verbose results from commands, detailed Git status information, lint findings, and other tool output all consume context.
- **Test output:** A test suite can produce a substantial log even when every test passes.
- **Build output:** Commands such as npm run build may generate long logs that occupy context without adding much useful information.

Understanding the cost of test output

Test output can consume much more context than a normal exchange. As a rough comparison, a conversation message may contain about 50 to 200 tokens, and a Codex response may contain about 500 to 1,000 tokens. A single test-suite run on a medium-sized project can produce roughly 2,000 to 5,000 tokens of output.

This means one test run can use as much context as five to ten complete message exchanges. We should still run tests when they are needed to verify our work, but we must account for their cost when planning a session.

Monitoring context with /status

Codex provides the built-in /status command to show information about the current session, including how much context has been consumed and how much remains. This gives us a practical way to check the context budget before it becomes a problem.

We should build the habit of running /status after operations that may be expensive, such as reading a large file or running a comprehensive test suite. Proactive checks let us adjust the session before context pressure affects the quality of the output.

Estimating the context cost of an operation

We can use four broad cost levels as a mental model:

- **Low cost:** Short messages, simple questions, single-line commands with minimal output, and small file reads.
- **Medium cost:** Medium-sized file reads, a linter run on a small project, a short test suite with clean output, or a small Git diff.
- **High cost:** Large source files, a full test suite, a verbose package installation, or a large Git diff that touches many files.
- **Very high cost:** Full project builds, tests with detailed coverage reports, multiple large files read in sequence, or a long correction cycle involving extensive codebase exploration.

© 2026 Zenva Pty Ltd. All rights reserved.

These categories are estimates rather than fixed measurements, but they help us anticipate which operations may consume a large portion of the available context.

Running expensive operations outside Codex

When Codex only needs an operation’s final result or one specific piece of information, we can run the operation externally and return only the relevant output.

- Run a full build outside the session, then provide only the errors that need investigation.
- Install packages externally, then tell Codex that the dependencies are available instead of adding the complete installation log to the context.
- For a long test suite, run the tests externally and provide only the failing output when the immediate goal is to diagnose failures.

The guiding principle is simple: if we need only the result or a small part of an operation’s output, we should avoid filling the context window with the entire verbose log. By recognizing hidden costs, checking /status, estimating operation costs, and externalizing appropriate tasks, we can reserve context for the reasoning and information that matter most.

© 2026 Zenva Pty Ltd. All rights reserved.

Managing Codex sessions well means deciding whether the context we have already built is still useful. The correct choice lets us preserve valuable understanding, remove irrelevant history, or restore earlier work without wasting time and tokens. We will compare four options: /compact, /new, codex resume, and a completely fresh Codex session.

Comparing the four session options

We can distinguish the options by asking whether they preserve the current conversation thread and whether they save a session for later:

- **/compact:** Preserves the current thread and saves it by compressing the conversation history.
- **/new:** Starts a clean conversation without preserving the current thread, but saves the previous session so we can return to it later.
- **codex resume:** Restores a saved conversation and its full history so we can continue the preserved thread.
- **A completely fresh session:** Provides a clean slate without preserving the current thread or carrying its session history forward.

Continuing with /compact

We should use /compact when the current task is still in progress and its context remains valuable. It is especially useful when the context window is becoming full or when Codex begins producing lower-quality results because the conversation has grown too long.

Compaction summarizes and condenses earlier parts of the conversation. This frees context-window space while keeping the current thread active, allowing us to continue the same task with its essential history intact.

We should not compact after switching to a genuinely different task. Doing so carries irrelevant context into the new work. This context pollution adds noise and can reduce the quality of Codex's responses.

Changing topics with /new

We should use /new when moving to an unrelated task and wanting a clean conversation without leaving the terminal. The command clears the current conversation history from the active thread, but saves the previous session in the background so it can be retrieved later.

We should not use /new in the middle of an ongoing task. Starting a new conversation at that point discards the hard-won understanding we have built with Codex. We would then need to spend additional time and tokens reconstructing that context from the beginning.

Restoring work with codex resume

We should use codex resume to continue a previously active session exactly where we left it. This is useful when work continues across different days or when a session was closed before its task was finished.

The command restores the conversation history and all accumulated context. For a faster return to the most recent session, we can use codex resume --last.

We should avoid resuming a session when its task is complete and we are beginning unrelated work. Restoring a finished or irrelevant session introduces dead context. Codex must process unnecessary information, which clutters the conversation and adds noise to new prompts.

© 2026 Zenva Pty Ltd. All rights reserved.

Starting a completely fresh session

A completely fresh session is appropriate when existing context is harmful rather than helpful. We should consider a fresh start in the following situations:

- The current session has entered a correction spiral.
- The conversation has accumulated too much bad signal or unreliable assumptions.
- We can no longer trust the output produced within the session.
- We are beginning a genuinely new project or a major new phase.
- We want to test a different prompting strategy without influence from earlier conversation history.

Starting fresh is the wrong choice when a saved session already contains useful context for unfinished work. In that case, rebuilding the same understanding creates unnecessary time and token costs, especially for a complex or long-running task.

A practical decision framework

The key question is: does the context in the current or saved session still have value for the work we are about to do? We can apply the following framework:

1. **The task is in progress and its context remains highly relevant:** Use /compact when more context-window space is needed, then continue the same task.
2. **We are switching to an unrelated task but might return later:** Use /new to begin with a clean conversation while saving the previous session.
3. **We closed a session and want to continue where we stopped:** Use codex resume, or codex resume --last for the most recent session, to restore its valuable history.
4. **The session went badly or its context is irrelevant to a genuinely new project:** Start a completely fresh Codex session.

The costs of a wrong session decision

Incorrect choices affect both efficiency and output quality. The four common mistakes correspond directly to the four session options:

- **Irrelevant context pollution:** Using /compact when /new was appropriate carries old-task noise into unrelated work.
- **Lost valuable in-progress context:** Using /new when /compact was appropriate discards useful understanding and forces us to rebuild it.
- **Dead context clutter:** Using codex resume for a finished or failed task restores irrelevant information that distracts from current objectives.
- **Redundant context rebuild:** Starting fresh when we should resume makes us spend time and tokens re-establishing context that Codex had already saved.

By evaluating the value of existing context before acting, we can choose the session option that protects useful work, removes noise, and keeps Codex focused on the task at hand.

© 2026 Zenva Pty Ltd. All rights reserved.

A focused Codex session begins before we open the editor or send the first prompt. By treating session scope as a deliberate design decision, we can give each coding session a clear purpose, keep the context relevant, and produce work that is easier to review and maintain.

Design session scope deliberately

Session scope should not emerge accidentally as we work. If we open Codex without a defined objective, the task can grow from one concern into several unrelated concerns. This sprawl pollutes the context needed for later tasks and can reduce the quality of Codex's output.

A well-scoped session takes the opposite approach. Before the first prompt, we choose one clear concern and limit the work to that concern. This gives Codex a focused context and helps us avoid distractions that do not contribute to the intended result.

Use the one-sentence session test

The one-sentence session test defines the boundary of our work before we begin coding. We can apply it in three steps:

1. **State the goal:** Express the session's core purpose in one clear sentence before opening Codex.
2. **Check for multitasking:** Look for multiple unrelated concerns joined together in the same goal.
3. **Rescope when necessary:** If the sentence contains several concerns, choose one and defer the others to separate sessions.

For example, a goal such as "add filters and fix the form" combines two different concerns. Even if both tasks are useful, handling them together creates an unfocused session. Choosing either the filter work or the form fix gives the session a clearer purpose and cleaner context.

Decide what belongs in the session

A narrow goal does not mean that every supporting activity is out of scope. Some tasks directly serve the primary objective and therefore belong in the same session:

- **Directly related research:** We may need to investigate technical details or library-specific syntax required to solve the primary problem.
- **Focused sub-agent tasks:** We can delegate the collection of specific information that helps us move the main task forward.
- **Verification:** Tests and other checks belong because they confirm that we have completed the session's objective.

Other activities distract from the goal and should normally be deferred:

- **Unrelated bug fixes:** We can record a newly discovered bug without fixing it during the current session.
- **Tangential feature additions:** A small or loosely related improvement is still scope creep if it does not support the primary objective.
- **General documentation cleanup:** Tidying a README usually belongs in a separate session unless documenting the current change is an explicit requirement.

This distinction keeps the context clean while still allowing the research, delegation, and verification needed to complete the main task properly.

Use Git history to assess session discipline

© 2026 Zenva Pty Ltd. All rights reserved.

Our Git log provides evidence of how well previous sessions were scoped. Poorly scoped sessions tend to produce large commits that cover several concerns and use vague messages such as “various fixes and updates,” “more work,” or “cleanup.” These commits make it difficult to understand how the project evolved or to revert one specific change.

Well-scoped sessions produce focused, descriptive commits. Each commit addresses one clear concern, creating a precise project narrative. This makes changes easier to review, simplifies future maintenance, and allows us to revert specific work without affecting unrelated changes.

Build a compounding habit

Strong Codex users treat every session as a deliberate unit of work with a clear beginning, a defined scope, and a definite end. Repeating this approach creates a compounding benefit: focused sessions lead to focused commits, and focused commits build a readable, maintainable project history.

A clear history helps us understand past changes, revert targeted work, and continue development without rebuilding the entire context from scratch. A few seconds spent designing the session can save hours otherwise lost to context switching, debugging confusion, and maintenance problems.

Key takeaways

- Treat session scope as a deliberate design decision rather than an accidental result of the workflow.
- Define the session’s boundary with a one-sentence goal before opening Codex.
- Keep supporting research, focused delegation, and verification in scope, while deferring unrelated fixes and additions.
- Read the Git log to diagnose past session discipline and improve future working habits.

© 2026 Zenva Pty Ltd. All rights reserved.

Best Practices for Agentic Coding with Codex - Context and Session Control

1. A test suite prints thousands of lines, but you only need the failures. How should you protect context?

a. Run it outside Codex and provide only the failures.
b. Run it repeatedly inside the same session.
c. Paste the complete output into a new prompt.
d. Skip testing to preserve context.

2. Your current task continues, but its context is nearly full. Which option fits best?

a. Use /compact and continue the task.
b. Use /new and discard the active thread.
c. Resume a completed, unrelated session.
d. Start an unrelated task in the same context.

3. Your session goal is “add filters and fix an unrelated form bug.” What should you do?

a. Split the concerns into separate focused sessions.
b. Keep both tasks to reduce session count.
c. Add documentation cleanup to the same session.
d. Let the scope grow as new issues appear.

© 2026 Zenva Pty Ltd. All rights reserved.

An AGENTS.md file becomes valuable when it actively changes how Codex works. In this lesson, we will distinguish passive project documentation from actionable instructions, apply the correction habit in a Todo app, examine how rules compound over time, and learn how to assess an AGENTS.md file in under 30 seconds.

Turn project context into active instruction

A common mistake is treating AGENTS.md as a description of the project. A file that only lists React, Vite, CSS Modules, and Vitest provides little new value because Codex can usually infer that information from files such as package.json. These details consume context without meaningfully guiding behavior.

Active instructions tell Codex what it cannot reliably infer from the code. They establish project policies, architectural constraints, and required verification steps. Useful rules might require all styling to use CSS Modules, prohibit installing dependencies without permission, require tests after changes to component logic, or state that task data must remain in App.jsx.

Effective rules share three qualities:

- They are specific enough to guide a concrete decision.
- They are actionable, so Codex knows exactly what to do or avoid.
- They prevent a known mistake or preserve an intentional project convention.

Vague statements such as “write clean code” or “follow best practices” do not create useful guardrails. An empty Critical Rules section is equally ineffective. We should fill that section with constraints that shape Codex’s output in ways the existing code cannot.

Build the correction habit

Correcting Codex without updating AGENTS.md creates a wasteful loop. The current problem disappears, but the same type of mistake can return in a later session. The correction habit replaces that loop with a two-step process:

1. Ask Codex to correct the immediate code or behavior.
2. Add a specific rule to AGENTS.md that prevents the mistake from recurring.

The first step solves one instance of the problem. The second step preserves the lesson for every future interaction with the project. Turning this process into a reflex is one of the highest-leverage improvements we can make to a Codex workflow.

Prepare a reversible demonstration

The demonstration uses a beginner-friendly Todo app built with React, Vite, and CSS Modules. Before making changes, we initialize Git and commit the existing project. This gives us a clean, reversible baseline and makes the agent’s changes easy to review.

© 2026 Zenva Pty Ltd. All rights reserved.

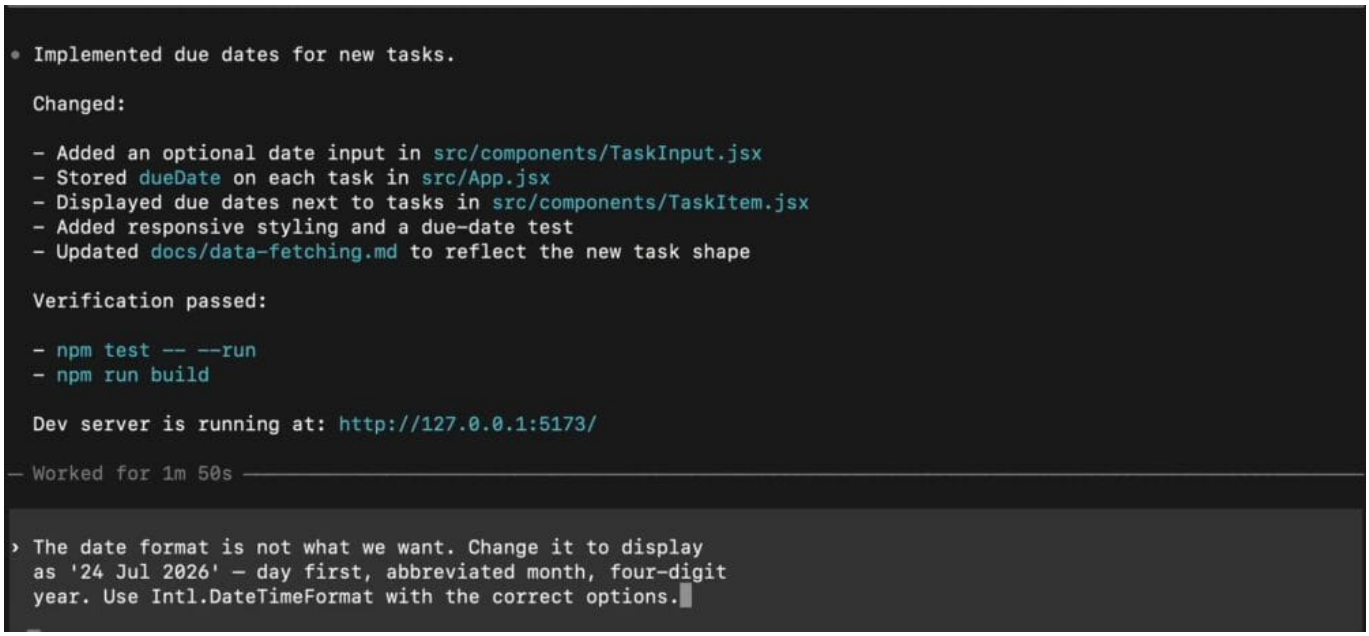
```
git add .
git commit -m "initial commit — todo app starter"
Initialized empty Git repository in /Users/devisl
[main (root-commit) d61001f] initial commit — tod
 25 files changed, 5097 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 AGENTS.md
 create mode 100644 README.md
 create mode 100644 docs/data-fetching.md
 create mode 100644 docs/ui.md
 create mode 100644 eslint.config.js
 create mode 100644 index.html
 create mode 100644 package-lock.json
 create mode 100644 package.json
 create mode 100644 src/App.jsx
 create mode 100644 src/App.module.css
 create mode 100644 src/__tests__/tasks.test.jsx
 create mode 100644 src/components/FilterBar.jsx
 create mode 100644 src/components/FilterBar.modu
 create mode 100644 src/components/TaskInput.jsx
 create mode 100644 src/components/TaskInput.modu
 create mode 100644 src/components/TaskItem.jsx
 create mode 100644 src/components/TaskItem.modul
 create mode 100644 src/components/TaskList.jsx
 create mode 100644 src/components/TaskList.modul
 create mode 100644 src/index.css
 create mode 100644 src/main.jsx
 create mode 100644 src/setupTests.js
 create mode 100644 vite.config.js
 create mode 100644 vitest.config.js
devisland@codepios-Mac-mini react_project % clea
```

© 2026 Zenva Pty Ltd. All rights reserved.

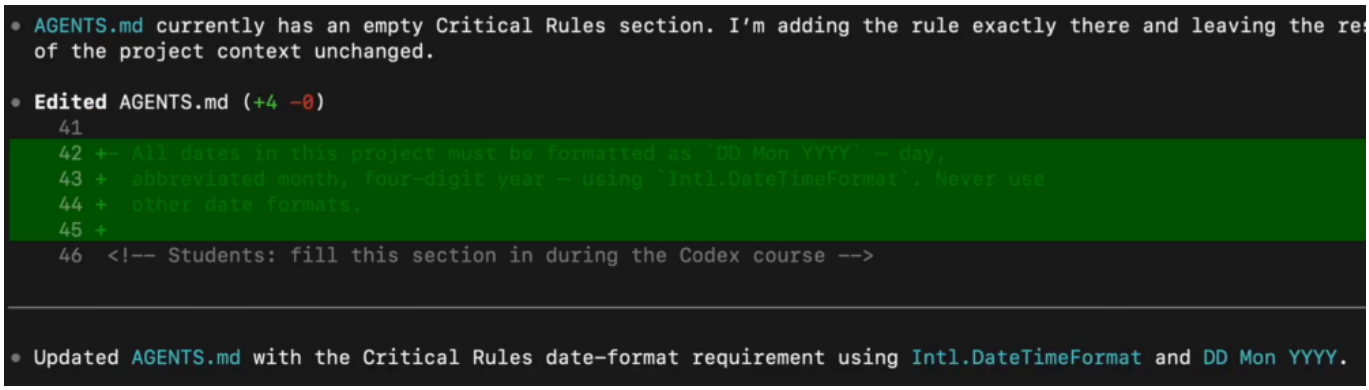
We then ask Codex to add an optional due date to the task input and display it beside each task. Codex examines the project structure, updates the relevant components, tests, and documentation, and verifies its work by running the tests and production build. It also requests permission before starting the local development server.

Correct the date format and preserve the rule

The first implementation adds the feature successfully, but its date format does not match the project's desired presentation. We provide a precise correction: display the day first, use an abbreviated month, include a four-digit year, and implement the format with Intl.DateTimeFormat. The expected result is a date such as 24 Jul 2026.

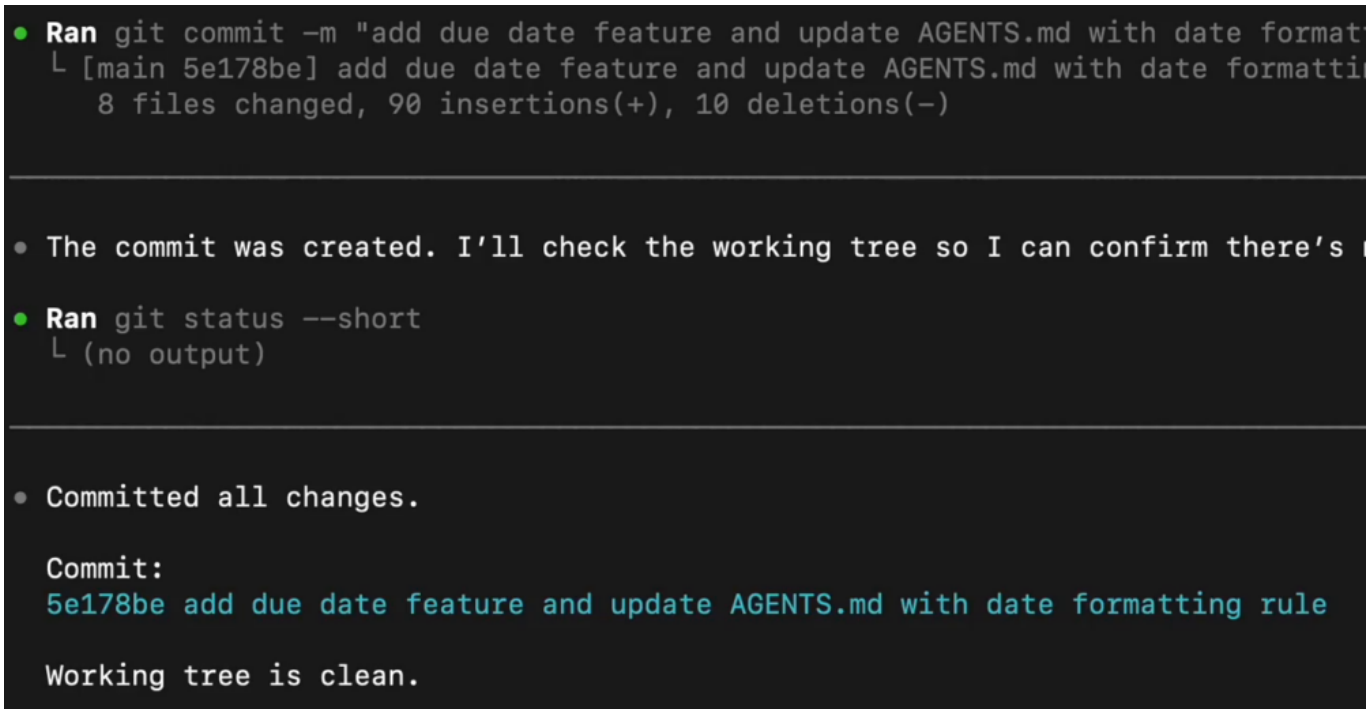


Codex updates the affected files and verifies the correction. At this point, stopping would leave the project vulnerable to the same formatting mistake later. We therefore apply the second half of the correction habit and ask Codex to add the convention to the Critical Rules section of AGENTS.md. The new instruction requires project dates to use the DD Mon YYYY pattern through Intl.DateTimeFormat and prohibits other formats.



Finally, we ask Codex to commit the due date feature and the new rule together using a specific commit message. Codex creates the commit, checks the repository status, and confirms that the working tree is clean. The code correction and the lasting project instruction now form one complete, traceable change.

© 2026 Zenva Pty Ltd. All rights reserved.



Let project knowledge compound

The correction habit becomes more valuable through consistent use. After the first correction, AGENTS.md contains one rule that prevents one mistake. After a week, the Critical Rules section may contain three to five rules, and repeated corrections begin to disappear.

After a month, the file may contain 10 to 15 rules covering common project-specific mistakes. Codex's first attempts become more accurate, and sessions finish with fewer corrections. After three months, AGENTS.md becomes a record of accumulated project learning. New developers can inherit the same guardrails immediately instead of rediscovering every convention through trial and error.

Diagnose AGENTS.md in under 30 seconds

We can quickly evaluate whether an AGENTS.md file is working by asking three questions:

- **Is the Critical Rules section empty?** If it is, the file is not actively preventing mistakes.
- **Are the rules vague?** Aspirational phrases do not tell Codex which concrete action to take.
- **Are we correcting the same type of mistake more than once?** If so, the missing fix is probably not in the code. It is a missing rule in AGENTS.md.

Strong rules use direct, testable language. Instructions such as never using inline styles or keeping state in App.jsx identify a precise behavior and remove ambiguity.

Key takeaways

- Treat AGENTS.md as active instruction, not a passive description of facts Codex can infer.
- Correct the immediate mistake, then encode the lesson as a specific rule.
- Allow rules to accumulate into shared project memory and stronger first-attempt results.
- Check for an empty Critical Rules section, vague guidance, and repeated corrections to find weaknesses quickly.

© 2026 Zenva Pty Ltd. All rights reserved.

AGENTS.md files give us precise control over the instructions Codex follows. By placing each rule at the correct level, we can keep personal preferences separate from shared project conventions and apply specialized guidance only where it is needed.

The three levels of the AGENTS.md hierarchy

The hierarchy has three levels: global, project, and subfolder. Each level has a distinct scope and purpose.

- **Global:** Personal Codex preferences that apply to every project.
- **Project:** Shared rules for one codebase.
- **Subfolder:** Context-specific rules for a directory and its contents.

Global configuration

The global AGENTS.md file is our personal Codex configuration. It is located at ~/.codex/AGENTS.md and applies universally to every project we open with Codex.

This level is the right place for workflow and communication preferences that should travel with us across all our work. Examples include asking Codex to explain what changed, prefer explicit variable names, or always run tests.

Global configuration is not the right place for project conventions or team agreements. Those belong at the project level.

Project configuration

A project-level AGENTS.md file typically lives at the root of a Git repository. It contains rules that are specific to that codebase, such as:

- The project’s technology stack, such as React 19 and Vite 7.
- Architectural decisions, such as keeping all state in App.jsx.
- Critical team rules, such as requiring approval before installing a dependency.

Because this file is committed to Git, every developer begins with the same Codex configuration. It becomes the project’s authoritative source for shared stack details, conventions, architectural decisions, and team rules.

Subfolder configuration

A subfolder-level AGENTS.md file applies only to its directory and the files and subdirectories it contains. This provides targeted control for specific parts of a codebase.

For example, a repository can have a project-level file at its root, then separate files for frontend, backend, or packages/ui-components. This is especially useful in monorepos or large, layered projects where different areas use different technologies or conventions. A React frontend and a Python backend in the same repository can therefore receive different guidance.

Subfolder configuration is valuable at scale, but it may not be necessary for a smaller project that uses a single framework throughout.

Common hierarchy mistakes

Two common placement mistakes can cause incorrect behavior and frustration.

© 2026 Zenva Pty Ltd. All rights reserved.

1. **Putting personal preferences in the project file:** When personal workflow habits are committed to the project's AGENTS.md, they are imposed on the entire team. Personal habits should remain in the global file.
2. **Putting project rules in the global file:** A convention that makes sense for one codebase, such as a particular state-management pattern, can produce incorrect guidance when applied universally. Codebase-specific rules should remain at the project level.

To avoid these mistakes, we should ask whether a rule applies everywhere or only within a particular project.

How conflicting rules are resolved

When rules at different levels conflict, the more specific rule takes precedence. This ensures that the instructions most relevant to the current context guide Codex.

1. The global AGENTS.md provides universal defaults.
2. The project AGENTS.md overrides global rules within that codebase.
3. A subfolder AGENTS.md overrides project rules within its directory scope.

In short, the most targeted rule always wins.

A practical placement checklist

We can decide where a rule belongs by working through four questions:

1. **Is this a personal workflow preference?** If it describes how we work regardless of the project or team, it belongs in the global file.
2. **Is this specific to the codebase?** If it concerns conventions, architecture, or technology choices unique to one project, it belongs at the project level.
3. **Is this relevant only to one part of the codebase?** If a directory has distinct requirements or a different technology stack, the rule belongs in a subfolder file.
4. **Would the team agree that this rule applies?** If yes, it is likely a shared project convention. If not, it is probably a personal preference for the global configuration.

Using this hierarchy gives us clear boundaries: personal preferences stay global, shared codebase rules stay with the project, and highly specific guidance stays in the relevant subfolder. When conflicts occur, specificity determines precedence.

© 2026 Zenva Pty Ltd. All rights reserved.

Scaling Codex across a team requires us to treat project knowledge as shared infrastructure. By storing the right configuration in version control, we can give every developer consistent guidance, improve onboarding, and keep important decisions from becoming isolated in individual workflows.

Separate shared knowledge from personal configuration

A common mistake is treating AGENTS.md as a personal file. When project conventions remain on one developer's machine, other team members must rediscover the same decisions and best practices. A project-level AGENTS.md should therefore be committed to version control like any other critical project configuration.

However, not every Codex setting belongs in the repository. We should separate personal workflow preferences from configuration that helps the whole team.

- **Keep personal:** Global AGENTS.md defaults, preferred model and approval settings, local sandbox preferences, and custom skills that only support an individual's workflow.
- **Share with the team:** Project-specific AGENTS.md rules, automated hooks, shared team skills, and consistent project sandbox settings.

This separation respects individual working styles while ensuring that every team member receives the same project-specific context.

Improve onboarding with inherited project context

Without shared Codex configuration, a new developer can clone the repository but still lack the project's conventions, architectural decisions, and critical rules. They may repeat old mistakes, spend time rediscovering established practices, or interrupt senior developers for information that should already be documented.

When the relevant configuration is stored with the project, a new team member inherits the team's accumulated knowledge as soon as they clone the repository. This reduces knowledge silos and helps new developers become productive more quickly.

Review AGENTS.md changes through pull requests

AGENTS.md should evolve with the codebase. Integrating its updates into the pull request process provides three important benefits:

- **Accurate and timely knowledge:** Updating rules alongside the code changes that require them prevents missing or outdated documentation.
- **Early team alignment:** Pull request review gives team members a place to discuss and resolve disagreements before a new rule silently influences Codex's behavior.
- **Active maintenance:** Regular review makes configuration maintenance a normal part of development and keeps AGENTS.md functioning as a living document.

Use a focused review checklist

When reviewing a proposed rule, we should ask three questions:

1. **Is the rule accurate?** It should describe the intended convention or decision precisely. Vague or incorrect guidance can cause unexpected behavior and wasted effort.
2. **Is the rule in the right place?** Personal preferences belong in global configuration, while project-specific conventions belong in the team's shared project file.
3. **Is the rule still relevant?** As the project changes, obsolete rules should be removed. Outdated guidance can be just as harmful as missing guidance.

© 2026 Zenva Pty Ltd. All rights reserved.

By sharing project configuration, supporting onboarding through inherited context, and reviewing AGENTS.md with every relevant pull request, we can keep the team’s collective memory accurate, useful, and available to everyone.

© 2026 Zenva Pty Ltd. All rights reserved.

Best Practices for Agentic Coding with Codex - Project Guidance That Scales

1. Codex repeats an incorrect date format. After fixing it, what best prevents recurrence?

a. Add a specific date-format rule to the project AGENTS.md.
b. Add a vague instruction to follow best practices.
c. List the project's technology stack again.
d. Rely on the current session to remember.

2. A monorepo’s Python backend needs rules that should not affect its React frontend. Where should they go?

a. An AGENTS.md inside the backend folder
b. The global personal AGENTS.md
c. The repository root only
d. An AGENTS.md inside the frontend folder

3. How should a team maintain shared project guidance?

a. Commit project AGENTS.md and review changes through pull requests.
b. Keep all rules on one developer's machine.
c. Put team rules in each person's global file.
d. Avoid updating guidance as the code changes.

© 2026 Zenva Pty Ltd. All rights reserved.

AI-assisted development becomes more reliable when verification is part of the work itself, not a separate activity that happens later. In this lesson, we will examine why visual inspection is insufficient for AI-generated code, compare long and collapsed feedback loops, and build a practical verification reflex into our Codex workflow.

Why code that looks right can still fail

We review AI-generated code differently from code we wrote ourselves. When reviewing our own work, we can retrace the decisions and intent behind each change. When Codex produces the code, our review is closer to acceptance testing because we do not have the same internal model of how every line was created.

This difference makes subtle problems harder to identify. Codex can present plausible code with confidence, but that presentation is not evidence of correctness. A quick visual scan may miss edge cases or interactions that only become visible when the code runs.

The strongest evidence comes from running the code against a mechanism that can fail. Tests, type checks, linting, and builds can expose concrete problems. Visual inspection remains useful for understanding intent and reviewing design choices, but it cannot prove that the implementation works.

Two common verification failures

The first failure pattern is skipping verification entirely. Codex produces plausible output, we accept and commit it, and a critical bug appears later in the workflow. We can prevent this by treating verification as a required step before accepting generated changes every time.

The second failure pattern is waiting until the Codex session has ended. Even when we eventually run checks, delayed verification creates a long feedback loop:

1. Codex generates code from the current prompt and project context.
2. We visually inspect the result and accept or commit the changes.
3. Later, we run tests, linting, or a full build for the first time.
4. A failure appears after the original task and reasoning are no longer fresh.
5. We open a new Codex session and explain the problem again.
6. Codex must reread files and reconstruct the missing context before debugging.
7. Codex proposes a fix, which may require further rounds of checking and correction.

This cycle consumes time and mental energy. Because the original generation context has been lost, both the developer and Codex must do additional work to understand the failure.

Collapse the feedback loop

In-session verification shortens this process. Codex writes the requested code and immediately runs the relevant checks while the full task context is still active. If a check fails, Codex can investigate, apply a fix, and run the check again. This continues until the required checks pass.

Only after verification do we review the final diff and Codex’s explanation. We can then accept the changes with stronger evidence that they meet the project’s existing quality standards. Keeping generation, testing, debugging, and review in one session reduces context loss and makes each iteration faster.

This collapsed feedback loop changes the economics of AI-assisted development. Earlier failures are cheaper to fix, fewer sessions are spent rebuilding context, and reliable output reaches completion faster.

© 2026 Zenva Pty Ltd. All rights reserved.

Build verification into the Codex workflow

We can make verification consistent through three approaches:

- **Prompt instructions:** Include the required command and expected result in each task prompt, such as asking Codex to run the tests and confirm that they pass. This is simple and explicit, but it depends on us remembering the instruction every time.
- **AGENTS.md rules:** Add a project rule such as always running tests after changes to component logic. Codex can then apply the requirement automatically across relevant tasks. This reduces reliance on memory, although other context can still affect how an instruction is interpreted.
- **Workflow hooks:** Use stop or post-tool-use hooks to trigger verification outside the agent loop. Hooks provide the most consistent option because they do not depend on an individual prompt or an instruction being recalled during the session.

The goal is to make proof part of the definition of done. Codex should not only claim that a change is correct. It should run the available checks and report the results.

Choose the right verification level

Different checks provide different kinds of confidence. We should use the project's existing tools and prioritize them according to what they can prove:

1. **Tests:** Tests provide the highest value when they are available because they catch logic errors and verify expected behavior.
2. **Type checking:** For TypeScript projects, a command such as tsc --noEmit catches type errors before runtime and verifies that API contracts are followed.
3. **Linting:** Linters catch code-quality issues, enforce style conventions, and prevent common mistakes with a relatively fast check.
4. **Build checks:** A production build confirms that the project compiles. It can reveal import errors, missing exports, and configuration problems that would prevent the application from starting.

Not every task requires every possible check, but each task should receive enough verification for the risk involved. A small change may need a focused test or build, while a broader change may justify the complete test suite. By selecting the appropriate checks and running them before the session ends, we gain better quality, faster feedback, and greater confidence in AI-assisted development.

© 2026 Zenva Pty Ltd. All rights reserved.

AI-generated code deserves the same careful review as a human contribution. In this lesson, we will build a repeatable process for examining Codex changes, combining a fast automated review with a deliberate manual inspection. This helps us understand every change, catch gaps that tests may miss, and protect the long-term quality of the codebase.

Avoid rubber-stamping AI output

Passing tests are important, but they are not enough to prove that a change is correct. Accepting generated code without reading it can allow subtle logic errors, architectural violations, silent dependency additions, or unrelated changes into the project. We remain responsible for the code we commit, regardless of whether it was written by a person or generated by Codex.

Every deep review should answer three questions:

1. **Did Codex touch the right files?** Start with the complete file list. Unexpected changes to configuration, unrelated components, or tests may reveal scope creep or a misunderstood task.
2. **Does the approach make sense?** Read the implementation closely and confirm that it addresses the root cause, follows project conventions, and handles relevant edge cases.
3. **Is anything missing?** A diff only shows what changed. We must also consider missing tests, outdated documentation, and related functions that may require the same update.

Use /review and git diff together

The /review command asks Codex to assess its own uncommitted changes. It is a fast first pass that can identify obvious mistakes, incomplete work, or conflicts with project conventions. It is useful for improving the change before we begin the final inspection.

```
/review
```

git diff serves a different purpose. It gives us exact before-and-after context for tracked files, allowing us to examine every changed line ourselves. This process is slower and requires concentration, but it is where human experience can uncover logical flaws, architectural conflicts, and missing work. The two tools are complementary: /review provides an automated sanity check, while git diff supports the final decision.

Apply the workflow to the Todo app

Before asking Codex to modify the Todo app, we create a clean Git baseline:

```
git init
git add .
git commit -m "initial commit - todo app starter"
```

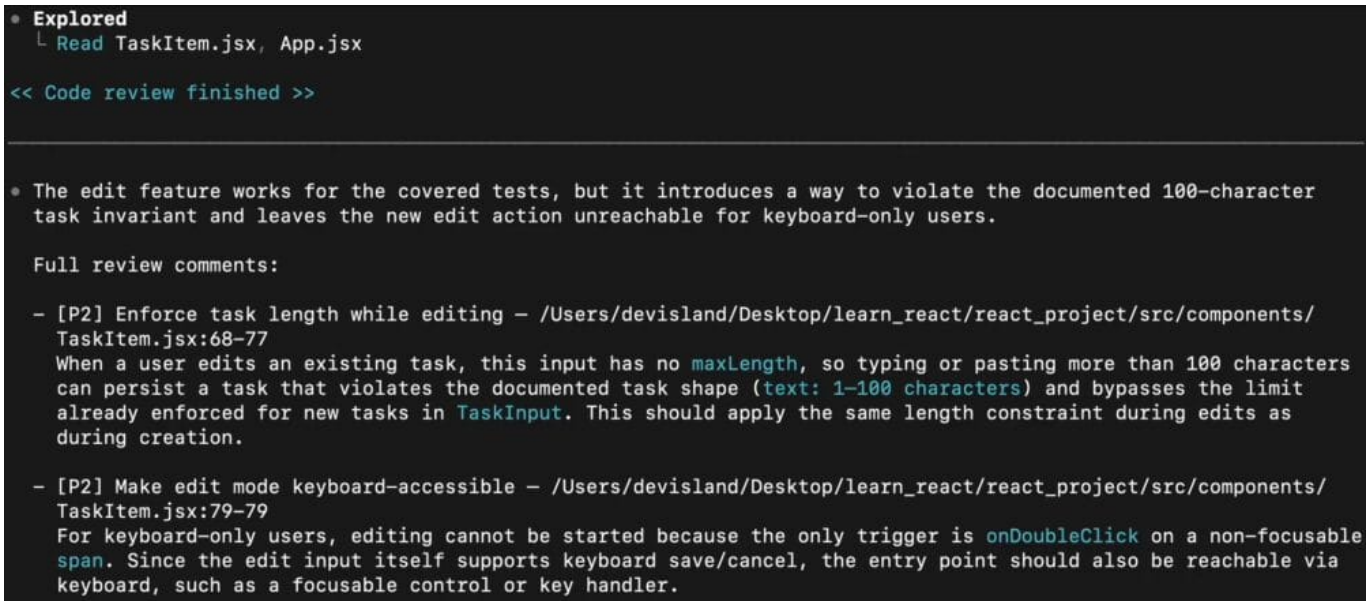
We then ask Codex to implement inline task editing with defined UI behavior. The change affects several files because the edit callback lives in App.jsx, passes through TaskList.jsx, and is used by TaskItem.jsx. The central state update added to App.jsx is:

```
const editTask = (id, text) => {
  setTasks(prev =>
    prev.map(task =>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
      task.id === id ? { ...task, text } : task
    )
  )
}
```

Codex also runs the tests and handles an edge case before summarizing its work. We then run /review. The automated review catches two issues that passing tests alone did not expose: edited tasks can exceed the documented 100-character limit, and a double-click-only edit trigger is inaccessible to keyboard-only users.



After the automated pass, we take control of the final inspection:

```
git status
git diff
```

We inspect the full file list, read each relevant diff, and consider the architectural effects of the new callback and editing state. Once the changes meet our expectations, we commit them and confirm that the Git work tree is clean.

Read a diff systematically

Simply scrolling from the top of a diff to the bottom is not an effective review strategy. We can make the process more reliable with a deliberate reading order:

1. **Check status first.** Use git status to see modified, added, and deleted files. This provides the full scope and helps expose unexpected changes.
2. **Prioritize critical files.** Review core business logic, shared utilities, configuration, and the files named in the task before less important changes.
3. **Read removed lines before added lines.** Understanding what disappeared gives us the context needed to judge whether the replacement is an improvement or whether important behavior was lost.
4. **Scrutinize shared code.** A small mistake in a utility, hook, or configuration file can affect

© 2026 Zenva Pty Ltd. All rights reserved.

many features, giving it a much larger blast radius than a localized UI change.

Inspect untracked files directly

Standard git diff does not display new untracked files. If Codex creates a component, utility, test, or configuration file, that file can remain invisible during a diff-only review. This is why git status must come first.

For every untracked file, open it in the editor or inspect it directly in the terminal:

```
cat path/to/new-file
```

We should read the entire file before staging or committing it. Otherwise, generated code could enter the repository without ever appearing in the tracked-file diff.

Follow the complete review flow

A professional review can be summarized as four steps:

1. Run /review to catch obvious errors and convention violations quickly.
2. Run git status to understand the complete set of changes, including untracked files.
3. Run git diff and inspect tracked changes in a deliberate priority order.
4. Open every new untracked file directly and review its full contents before committing.

Throughout the process, we can treat Codex output like a pull request from a junior developer. This is not about distrusting AI. It is a practical quality standard that combines scrutiny, mentorship, and responsibility. Our knowledge of the product and architecture remains essential for finding edge cases, missing tests, documentation gaps, and long-term design problems.

A focused review usually takes only a few minutes, but it can prevent hours or days of debugging and refactoring. By making this process habitual, we become the final quality gate and can ship AI-assisted changes with confidence.

© 2026 Zenva Pty Ltd. All rights reserved.

When a Codex session starts going wrong, the most important decision is whether to keep correcting it or reset and begin again. Making that choice quickly prevents wasted effort, keeps the task focused, and helps us avoid turning a small mistake into a complicated chain of partial fixes. In this lesson, we will learn how to recognize a correction spiral, identify the signals that favor a reset, and distinguish them from situations where one precise correction is likely to work.

Recognizing a correction spiral

A correction spiral is a deteriorating pattern in which repeated attempts to improve an output create more problems instead of producing clear progress. It usually develops through four stages:

1. **Initial misstep:** Codex produces an output that is clearly wrong, misses an important requirement, or introduces an explicit error.
2. **Flawed correction:** Our first correction changes the result, but the output remains wrong or fixes one issue while creating another.
3. **Escalating complexity:** Later corrections solve recent problems but regress on earlier points. The result becomes a patchwork of partial fixes without meaningful overall improvement.
4. **Stagnant progress:** Each local fix makes the output more complicated without addressing the underlying cause, such as vague direction, polluted context, or a fundamentally incorrect approach.

Spotting this pattern early is valuable because a session can appear active while making no real progress. More changes do not necessarily mean that the output is getting closer to the intended result.

Signals that it is time to reset

A reset is appropriate when the foundation of the session has become unstable. Four warning signs help us make that decision:

- **The correction limit has been exceeded:** We have corrected the same session more than twice without clear improvement. Further incremental attempts are likely to continue the loop.
- **Every fix breaks something else:** Each correction solves one problem but immediately introduces a new flaw or brings back an earlier one.
- **Complexity grows without correctness:** Prompts and responses become longer and more intricate, yet the fundamental problem remains. We are adding layers of fixes instead of addressing the root cause.
- **We lose the North Star:** We no longer have a clear picture of the ideal output, or we begin second-guessing the original requirements. Once the objective becomes muddled, a clean restart is usually more effective than another correction.

Signals that we should persist

Not every mistake requires a full reset. Sometimes the session has a sound foundation and needs only one focused correction. We should consider persisting in the following situations:

- **The error is a specific detail:** The overall structure and approach are correct, but one identifiable item, such as a variable name or formatting choice, is wrong.
- **The session is converging on clarity:** Codex asks a clarifying question, we provide a clear answer, and the next output demonstrably incorporates that information. This shows that the work is moving in the right direction.
- **Our prompt caused the misunderstanding:** On review, we can see that the correction prompt was ambiguous or omitted an important detail. If Codex made a reasonable

© 2026 Zenva Pty Ltd. All rights reserved.

interpretation of incomplete instructions, a precise reprompt may be enough.

The central distinction is whether the problem is local or foundational. A local, clearly identifiable mistake supports one more targeted correction. A session with unstable assumptions, repeated regressions, or a lost objective supports a reset.

Performing a clean reset

When a reset is the right choice, we should make it genuinely clean rather than carrying broken work into the next attempt. The process has three parts:

1. **Revert the code:** Restore the codebase to the last clean commit and discard partially generated changes. This gives the next session a reliable starting point.
2. **Diagnose the failure:** Take a few minutes to determine why the previous prompt failed. Check whether the intended outcome was unclear, necessary context was missing, or important constraints were left unstated.
3. **Tighten the scope:** If the failed session attempted several things at once, make the new prompt focus on one well-defined task. A narrower task is harder to misinterpret.

From the project folder, git restore . restores tracked files in the current directory to their last committed state. The command git clean -fd removes untracked files and directories. Together, these commands return the project to a clean slate before we begin a new session.

Using the ten-minute rule

It can be difficult to judge a session objectively while we are actively trying to repair it. A simple time threshold helps override that in-the-moment bias: if we have spent more than ten minutes correcting the same output without clear convergence, we should reset.

This rule is not about giving up. It prevents a non-converging session from consuming more time, effort, and attention. It also gives us a consistent decision point instead of relying on frustration or guesswork.

Resetting as a professional decision

A reset does not mean that the task is impossible or that the tool has failed completely. It means that we recognize a problem in the current approach or prompt and choose to address its root cause. By resetting, we move away from repeatedly treating symptoms and return our attention to the brief, context, constraints, and scope that we can directly control.

The earlier we make this decision, the less time and cognitive load the failed approach costs us. Effective Codex use therefore depends on both persistence and restraint. We persist when a session is clearly converging and the remaining problem is specific. We reset when corrections create regressions, complexity grows without correctness, the objective becomes unclear, or the ten-minute threshold passes without progress.

© 2026 Zenva Pty Ltd. All rights reserved.

Advanced Codex tools can make our work faster, more reliable, and easier to scale, but each tool also introduces setup and coordination costs. In this lesson, we will build a practical framework for choosing among subagents, parallel sessions, hooks, and skills. The aim is to match each tool to the specific problem it solves while recognizing when a focused single session remains the better choice.

Match each tool to its core problem

Before choosing a tool, we should identify the cost or constraint that is limiting our current workflow:

- **Subagents** address the context cost of deep research. They let a separate agent investigate a codebase while the main session preserves its context for implementation.
- **Parallel sessions** address the time cost of sequential, independent work. They allow substantial tasks to progress simultaneously.
- **Hooks** address the reliability cost of manual enforcement. They automatically run important checks after relevant events.
- **Skills** address the repetition cost of recurring prompt patterns. They preserve proven workflows so we can invoke them again without reconstructing the prompt.

This distinction matters because advanced tools are specialized solutions. Their overhead is worthwhile only when it is smaller than the problem they remove.

Use subagents for substantial exploration

A subagent is useful when research would consume too much of the main session’s context. Strong candidates include tasks that require reading more than three or four files, exploring multiple directories, or developing significant codebase understanding before implementation begins. By separating research from implementation, we keep the main session focused and clear.

Simple lookups do not justify this setup. If we only need to inspect one file for a specific pattern, a direct reference in the main prompt is faster. A useful signal is a task that naturally divides into two phases: first explore the codebase and understand how something works, then implement a change. That structure suggests assigning the exploration phase to a subagent.

Run parallel sessions only for independent work

Parallel sessions create a meaningful speedup when tasks are genuinely independent. They should have no file overlap, no dependency between their results, and enough work to offset the coordination cost. As a rule of thumb, each task should take at least ten minutes.

Small tasks that take about five minutes rarely benefit from parallel execution because switching between sessions can consume the time we hoped to save. Before starting, we should be able to define each session’s ownership boundary in one sentence. For example, one session might own the components directory while another owns the services directory. If we cannot draw a clear boundary, the tasks are probably too connected to run safely in parallel.

Use hooks for checks that must always run

Hooks are appropriate when a quality check must happen consistently after a recurring event, such as every file write. They are especially valuable when prompts, manual reminders, or **AGENTS.md** rules cannot guarantee that the check will run every time.

A rare check does not justify a hook. For example, a full integration test suite run only once before a major release is too infrequent for the setup and maintenance cost to pay off. A practical trigger is repetition: if we have reminded Codex to run the same check more than three times in one project, that check is a strong hook candidate. Automating it removes the need for future reminders and

© 2026 Zenva Pty Ltd. All rights reserved.

creates reliable enforcement.

Turn recurring prompt patterns into skills

A skill is worth creating when a prompt pattern is effective, recurring, and stable. The workflow should already be proven reliable, likely to be needed again, and structured consistently enough that it will not require substantial edits with every use.

One-off tasks and highly variable workflows should remain ordinary prompts. A simple trigger is the second occurrence: when we write a nearly identical structured prompt for the second time, we should consider saving it as a skill. The third time the workflow is needed, we can invoke it by name instead of rebuilding it from scratch.

Apply the quick decision framework

- If research across more than three or four files would consume too much main-session context, use a **subagent**.
- If two independent tasks each require at least ten minutes and have zero file overlap, use **parallel sessions**.
- If the same important check has required more than three reminders, automate it with a **hook**.
- If we are writing the same reliable prompt structure for the second time, capture it as a **skill**.

Default to the simplest effective tool

For most tasks, a single session with a well-written prompt is the most efficient approach. We should not treat advanced tools as general improvements that belong in every workflow. They solve specific problems that a single session cannot handle efficiently.

Choosing either the simplest or the most complex option by default can create avoidable problems. Instead, we should identify the actual constraint, compare it with the tool’s overhead, and choose the simplest option that solves the problem effectively. This keeps the workflow clear and prevents unnecessary coordination.

Build workflow infrastructure that compounds

Each correct tool choice saves context, time, or mental effort. That benefit carries into later sessions by making the overall workflow more efficient. Hooks permanently remove recurring friction, while skills capture successful patterns and make them immediately reusable.

This approach is more than a collection of prompting techniques. It creates durable workflow infrastructure that automates enforcement, accelerates independent work, and preserves valuable context. As that infrastructure improves, we can devote more attention to the judgment and creativity that require human direction.

© 2026 Zenva Pty Ltd. All rights reserved.

Best Practices for Agentic Coding with Codex - Verification and Workflow Decisions

1. Codex says a feature is complete. Which workflow gives the strongest evidence before committing?

a. Run relevant checks, then review the diff and untracked files.
b. Trust the summary if the code looks plausible.
c. Commit first and test in another session.
d. Review only the files Codex mentions.

2. After two corrections, each fix causes another regression and the goal is becoming unclear. What should you do?

a. Reset cleanly, diagnose the cause, and narrow the brief.
b. Continue correcting until one version works.
c. Commit the partial result before trying again.
d. Add more tasks to rebuild context.

3. You have reminded Codex to run the same check more than three times. Which tool best fits?

a. A hook
b. A subagent
c. A parallel session
d. A one-off prompt

© 2026 Zenva Pty Ltd. All rights reserved.

We have developed a set of practical habits for working more effectively with Codex. Together, these habits improve how we diagnose problems, write prompts, manage context, preserve project knowledge, and review AI-generated work.

Diagnose failures with better mental models

When a session fails, we can look beyond the most obvious symptom and identify the underlying cause. The director versus executor pattern helps us determine whether the problem comes from the direction we provided or from how that direction was carried out. We can then identify which of the four failure modes applies and choose a precise fix for that cause.

Prompt with precision

Effective prompts include the right outcome, context, constraints, and examples. They also leave out information that does not help Codex complete the task. By placing specificity where it matters, we improve the quality of the first attempt and reduce the need for repeated corrections.

Design focused sessions

Context discipline begins before we start writing. We deliberately define a focused session scope and make conscious choices between compacting, clearing, or resuming a session. This treats session scope as an important design decision instead of an afterthought.

Maintain living project instructions

An **AGENTS.md** file is more than project documentation. We can maintain it as an active instruction set, turning mistakes into permanent rules and placing guidance at the appropriate level of the project hierarchy. This creates a memory system that can scale across an entire team.

Verify and review every result

Strong verification habits help us catch issues that automated tests can miss. Before accepting output, we verify it within the session, read every diff before committing, and review AI-generated code with the same scrutiny we would apply to a human-written pull request.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

AGENTS.md

Found in folder: **/**

This file provides project context and guidelines for developers working on the Todo app. It outlines the technology stack, available commands, and coding conventions. It also includes a placeholder section for critical rules to be added later.

```
# AGENTS.md

This file gives Codex context about the project.

## Project

A beginner-friendly Todo app built with React and Vite, used as a starter
project for a Codex course. Students extend the app by adding missing
features — guided by Codex.

## Stack

| Layer       | Choice                        |
|-------------|-------------------------------|
| UI          | React 19                      |
| Bundler     | Vite 7                        |
| Styling     | CSS Modules (plain CSS only)  |
| Tests       | Vitest + Testing Library      |
| Persistence | localStorage                  |

No backend. No routing. No auth. No external UI libraries.

## Commands

| Command           | Purpose                          |
|-------------------|----------------------------------|
| `npm run dev`     | Start dev server (localhost:5173)|
| `npm run build`   | Production build                 |
| `npm test`        | Run tests in watch mode          |
| `npm run test:ui` | Open the Vitest browser UI       |

## Conventions

- All state lives in `App.jsx`; components receive data and callbacks as props
- One component per file; its CSS module lives in the same directory
- Task shape: `{ id: string, text: string, completed: boolean }`
- See `/docs/ui.md` for styling rules
- See `/docs/data-fetching.md` for state and persistence rules

## Critical Rules
```

© 2026 Zenva Pty Ltd. All rights reserved.

README.md

Found in folder: /

This file is a project README that documents a Todo app starter kit. It lists existing features, missing functionality to be added as exercises, and provides setup instructions. The document also outlines the project structure and technology stack used.

````
# Todo App — Claude Code Course Starter

A deliberately incomplete Todo app. You will use **Claude Code** to add the
missing features listed below.

## Getting started

```bash
npm install
npm run dev        # http://localhost:5173
```

## Running tests

```bash
npm test           # watch mode
npm run test:ui    # browser UI
```

One test passes. One test fails — that is intentional. Fixing it is your
first exercise.

## What already works

| Feature | File |
|---|---|
| Add a task | `TaskInput.jsx` |
| Mark complete (checkbox) | `TaskItem.jsx` |
| Delete a task | `TaskItem.jsx` |
| Character counter (max 100, turns red) | `TaskInput.jsx` |
| Filter bar — All / Active / Completed | `FilterBar.jsx` |
| localStorage persistence | `App.jsx` |

## What is missing (your exercises)

- **Empty-input validation** — clicking Add Task with a blank field still
  creates an empty task. Add a guard in `TaskInput.jsx` to prevent this.
  The failing test in `src/__tests__/tasks.test.jsx` will go green once you do.

- **Task editing** — double-clicking a task text should let you edit it
  in place and save on Enter / blur.
````

© 2026 Zenva Pty Ltd. All rights reserved.

```
- **Due dates** — tasks have no deadline field. Add a date picker to
  `TaskInput` and display the date in `TaskItem`.

- **Drag to reorder** — tasks are fixed in insertion order.

## Project structure

```
src/
  components/
    TaskInput.jsx + TaskInput.module.css
    TaskList.jsx  + TaskList.module.css
    TaskItem.jsx  + TaskItem.module.css
    FilterBar.jsx + FilterBar.module.css
  App.jsx + App.module.css
  main.jsx
  index.css
docs/
  ui.md            — styling conventions
  data-fetching.md — state and persistence conventions
CLAUDE.md          — context file for Claude Code
vitest.config.js
```

Stack

- React 19 + Vite 7
- CSS Modules only (no Tailwind, no UI libraries)
- Vitest + Testing Library
- localStorage (no backend)
```

data-fetching.md

Found in folder: **/docs**

This markdown file documents the state management and data flow conventions for a todo app, describing the task object structure, persistence using localStorage, and how tasks are filtered and mutated. It explains that all state is centralized in App.jsx and outlines the callback functions for task operations. The document also notes current limitations and features not yet implemented.

```
# Data & State Conventions

## State Shape

All app state lives in `App.jsx`. There is no global state manager.

```js
// Task object — the single source of truth
{
  id: string,        // crypto.randomUUID(), set at creation time
  text: string,      // 1–100 characters (MAX_LENGTH enforced in TaskInput)
  completed: boolean // toggled by the checkbox
}
```

© 2026 Zenva Pty Ltd. All rights reserved.
```

```

The active filter (`'all' | 'active' | 'completed'`) is also stored in `App`
but is never persisted — it resets to `'all'` on page load.

## Persistence

Tasks are persisted to `localStorage` under the key `"todo-tasks"`.

**Read** — lazy initialiser so it runs exactly once on mount:

```js
const [tasks, setTasks] = useState(() => {
  const stored = localStorage.getItem('todo-tasks')
  return stored ? JSON.parse(stored) : []
})
```

Write — effect that syncs whenever `tasks` changes:

```js
useEffect(() => {
  localStorage.setItem('todo-tasks', JSON.stringify(tasks))
}, [tasks])
```

Task Operations

All mutations live in `App.jsx` and are passed to children as callbacks:

Prop	Signature	Behaviour
`onAdd`	`(text: string) => void`	Prepends a new task to the array
`onToggle`	`(id: string) => void`	Flips `completed` on one task
`onDelete`	`(id: string) => void`	Removes the task from the array

Mutations always use the functional updater form (`setTasks(prev => ...)`) to
avoid stale-closure bugs.

Filtering

Filtering is derived — it does **not** mutate `tasks`:

```js
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true
})
```

Only `filteredTasks` is passed to ``. The full `tasks` array is
used for the count in ``.

What Is Not Here (yet)
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- No API calls — everything is local; no `fetch`, no async state
- No loading or error states
- No optimistic updates
- No task editing, due dates, or ordering — those are student exercises in the Codex course


File extension: md
```

ui.md

Found in folder: **/docs**

This file documents the user interface design system and component structure for the project. It defines visual conventions like colors, spacing, and layout rules. It also outlines the component architecture and provides instructions for adding new components.

```
# UI Conventions

## Layout

- All components live in `src/components/`
- Each component has a matching `.module.css` file in the same directory
- Max content width: `600px`, centered via `margin: auto` in `App.module.css`
- Page background: `#f3f4f6` (set in `src/index.css`)

## Colour Palette

| Token        | Value     | Used for                          |
|--------------|-----------|-----------------------------------|
| Primary      | `#6366f1` | Buttons, focus rings, checkboxes  |
| Primary dark | `#4f46e5` | Button hover                      |
| Danger       | `#ef4444` | Delete hover, character limit     |
| Danger light | `#fee2e2` | Delete button hover background    |
| Border       | `#d1d5db` | Default input / pill borders      |
| Border inner | `#e5e7eb` | Task item border                  |
| Text muted   | `#9ca3af` | Empty state, count, delete icon   |
| Text base    | `#1a1a1a` | Body copy                         |
| Surface      | `#fff`    | Cards, inputs                     |

## Border Radii

| Element              | Radius |
|----------------------|--------|
| Inputs, buttons      | `6px`  |
| Task list items      | `8px`  |
| Filter pills         | `20px` |
| Delete button        | `4px`  |

## Interaction

- Every interactive element has a `transition` on whichever property changes
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- Default transition duration: `0.15s`
- Checkboxes use `accent-color: #6366f1`

## Component Hierarchy

```
App
??? TaskInput      — input field, Add Task button, character counter
??? FilterBar      — All / Active / Completed pills + task count
??? TaskList
    ??? TaskItem   — checkbox, task text, delete button
```

Adding a New Component

1. Create `src/components/ComponentName.jsx`
2. Create `src/components/ComponentName.module.css` alongside it
3. Import CSS with `import styles from './ComponentName.module.css'`
4. Apply class names with `className={styles.className}`
5. Wire it into the appropriate parent component; lift any new state to `App.jsx`
```

App.jsx

Found in folder: **/src**

This code manages a todo list app by storing tasks in local storage and allowing users to add, toggle, delete, and edit tasks. It filters tasks based on their completion status and displays them through child components. The app maintains state for tasks and a filter, with effects to persist tasks to local storage.

```
import { useState, useEffect } from 'react'
import TaskInput from './components/TaskInput'
import TaskList from './components/TaskList'
import FilterBar from './components/FilterBar'
import styles from './App.module.css'

function App() {
  // Initialize tasks from localStorage or empty array
  const [tasks, setTasks] = useState(() => {
    const stored = localStorage.getItem('todo-tasks')
    return stored ? JSON.parse(stored) : []
  })
  const [filter, setFilter] = useState('all')

  // Persist tasks to localStorage whenever they change
  useEffect(() => {
    localStorage.setItem('todo-tasks', JSON.stringify(tasks))
  }, [tasks])

  const addTask = (text) => {
    setTasks(prev => [
      { id: crypto.randomUUID(), text, completed: false },
      ...prev,
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    ])
  }

  const toggleTask = (id) => {
    setTasks(prev =>
      prev.map(task =>
        task.id === id ? { ...task, completed: !task.completed } : task
      )
    )
  }

  const deleteTask = (id) => {
    setTasks(prev => prev.filter(task => task.id !== id))
  }

  const editTask = (id, text) => {
    setTasks(prev =>
      prev.map(task =>
        task.id === id ? { ...task, text } : task
      )
    )
  }

  // Filter tasks based on current filter selection
  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed
    if (filter === 'completed') return task.completed
    return true
  })

  return (
```

Todo

```
  )
}

export default App

/*
Optional due-date reference (inactive):

const addTask = (text, dueDate) => {
  setTasks(prev => [
    { id: crypto.randomUUID(), text, dueDate, completed: false },
    ...prev,
  ])
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
*/
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM. It uses React's StrictMode to highlight potential problems during development. The App component is wrapped inside a root DOM element with the id 'root'.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

// Create a React root at the DOM element with id 'root' and render the app
createRoot(document.getElementById('root')).render(



)
```

FilterBar.jsx

Found in folder: **/src/components**

This code renders a set of filter buttons and a task counter. It maps over a list of filter types to create interactive buttons, highlighting the currently active one. It also displays the total number of tasks, using the correct singular or plural form.

```
import styles from './FilterBar.module.css'

// Predefined list of filter options
const FILTERS = ['all', 'active', 'completed']

export default function FilterBar({ filter, onFilterChange, taskCount }) {
  return (

      {FILTERS.map(f => (
         onFilterChange(f)}
        >
          {/* Capitalize the first letter of the filter name for display */}
          {f.charAt(0).toUpperCase() + f.slice(1)}

      ))}

        {/* Display count with correct pluralization */}
        {taskCount} {taskCount === 1 ? 'task' : 'tasks'}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  )
}
```

TaskInput.jsx

Found in folder: **/src/components**

This code provides a text input and button for adding new tasks, with validation and a character counter. It trims whitespace from the input before adding and prevents empty submissions. The component also includes visual feedback when approaching the character limit.

```
import { useState } from 'react'
import styles from './TaskInput.module.css'

const MAX_LENGTH = 100 // Maximum allowed characters for a task

export default function TaskInput({ onAdd }) {
  const [text, setText] = useState('')

  const handleAdd = () => {
    const nextText = text.trim()
    if (!nextText) return // Prevent adding empty tasks

    onAdd(nextText)
    setText('') // Clear input after adding
  }

  return (


                  className={styles.input}
          type="text"
          value={text}
          onChange={e => setText(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && handleAdd()} // Allow adding with Ente
r key
          placeholder="What needs to be done?"
          maxLength={MAX_LENGTH}
          aria-label="New task"
        />

          Add Task



      {/* Character counter with visual warning when approaching limit */}


= MAX_LENGTH ? styles.atLimit : ''}`}>
        {text.length} / {MAX_LENGTH}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  )
}

/*
Optional due-date reference (inactive):

const [dueDate, setDueDate] = useState('')

const handleAdd = () => {
  const trimmedText = text.trim()

  if (!trimmedText) return

  onAdd(trimmedText, dueDate || null)
  setText('')
  setDueDate('')
}

  className={styles.dateInput}
  type="date"
  value={dueDate}
  onChange={e => setDueDate(e.target.value)}
  aria-label="Due date"
/>
*/
```

TaskItem.jsx

Found in folder: **/src/components**

This component renders a single task item with a checkbox, editable text, and a delete button. It allows toggling completion, editing via double-click, and saving/canceling edits with Enter/Escape keys or blur events. The editing state manages focus and prevents unwanted save on certain interactions.

```
import { useEffect, useRef, useState } from 'react'
import styles from './TaskItem.module.css'

export default function TaskItem({ task, onToggle, onDelete, onEdit }) {
  const [isEditing, setIsEditing] = useState(false)
  const [draft, setDraft] = useState(task.text) // Local copy for editing
  const inputRef = useRef(null)
  const ignoreBlurRef = useRef(false) // Prevents save on blur for cancel actions

  // Focus and select text when entering edit mode
  useEffect(() => {
    if (isEditing) {
      inputRef.current?.focus()
      inputRef.current?.select()
    }
  }, [isEditing])

  const startEditing = () => {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    ignoreBlurRef.current = false
    setDraft(task.text)
    setIsEditing(true)
  }

  const saveEdit = () => {
    const nextText = draft.trim()
    if (nextText) {
      onEdit(task.id, nextText)
    }
    setIsEditing(false)
  }

  const cancelEdit = () => {
    ignoreBlurRef.current = true // Signal to ignore the upcoming blur event
    setDraft(task.text)
    setIsEditing(false)
  }

  const handleKeyDown = (e) => {
    if (e.key === 'Enter') {
      e.preventDefault()
      saveEdit()
    }

    if (e.key === 'Escape') {
      e.preventDefault()
      cancelEdit()
    }
  }

  const handleBlur = () => {
    if (ignoreBlurRef.current) {
      ignoreBlurRef.current = false
      return
    }

    saveEdit() // Save on blur unless canceled
  }

  return (

              type="checkbox"
        className={styles.checkbox}
        checked={task.completed}
        onChange={() => onToggle(task.id)}
        aria-
label={`Mark "${task.text}" as ${task.completed ? 'active' : 'complete'}`}
      />
      {isEditing ? (
                  ref={inputRef}
          className={styles.editInput}
          type="text"
          value={draft}
          onChange={e => setDraft(e.target.value)}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          onKeyDown={handleKeyDown}
          onBlur={handleBlur}
          aria-label={`Edit "${task.text}"`}
        />
      ) : (

          {task.text}

      )}
       onDelete(task.id)}
        aria-label={`Delete "${task.text}"`}
      >
        ?


  )
}

/*
Optional due-date reference (inactive):

const dueDateFormatter = new Intl.DateTimeFormat('en-GB', {
  day: 'numeric',
  month: 'short',
  year: 'numeric',
  timeZone: 'UTC',
})

function formatDueDate(dueDate) {
  return dueDateFormatter.format(new Date(`${dueDate}T00:00:00Z`))
}


  {task.text}
  {task.dueDate && (

      Due: {formatDueDate(task.dueDate)}

  )}

*/
```

TaskList.jsx

Found in folder: **/src/components**

This component renders a list of tasks. If the list is empty, it displays a message. Otherwise, it maps each task to a TaskItem component, passing down the task data and callback functions.

```
import TaskItem from './TaskItem' import styles from './TaskList.module.css' export
 default function TaskList({ tasks, onToggle, onDelete, onEdit }) { // Show a message
 if there are no tasks if (tasks.length === 0) { return
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
No tasks here.

 } // Render a list of TaskItem components for each task return (

        {tasks.map(task => (
```

© 2026 Zenva Pty Ltd. All rights reserved.